

Deep Repo Defense Guide

Private PDF study guide for defending Deep-Learning-On-OSC in a professor interview. This guide is grounded only in the local repository and local external clones present in the workspace.

Generated from /Users/jaredyoung/Documents/Programs/GitHub/Deep-Learning-On-OSC at 2026-04-07T23:15:01. Scope covered here: Q3 through Q7 plus repo-level orchestration and OSC helpers.

Repo Orientation

- This repository is primarily a wrapper/orchestration repo, not a repo that reimplements every deep-learning backbone from scratch.
- The clearest division is at `external/`: FasterRCNN, Ultralytics, YOLOv12, and TLib live there; the tracked files in the repo root and `question_*` folders normalize environment setup, path handling, GPU checks, resumability, and artifact layout.
- The code gets progressively more wrapper-heavy in Q3, Q4, Q5, and Q7, while Q6 is the most self-contained original implementation.

Shared patterns to defend across the whole repo

- Python 3.9.18 is enforced repeatedly across question folders and root orchestrators, which means interpreter selection is part of the repo's design, not an incidental detail.
- OSC GPU access is treated as an infrastructure concern handled by Slurm wrappers and preflight helpers rather than by the per-question Python scripts.
- Most question folders separate three concerns: setup, execution, and artifact management. That pattern is explicit in filenames and comments.
- The repo often chooses inline compatibility shims or wrapper subclasses instead of editing upstream projects directly. Q3 and Q5 are the clearest examples.

External backbone map

- `external/FasterRCNN` -> `trzy/FasterRCNN`; local clone head: 1ab2d3c.
- `external/ultralytics` -> `ultralytics/ultralytics`; local clone head: 8d4e6e8.
- `external/yolov12` -> `sunsmarterjie/yolov12`; local clone head: d3cbe10.
- `external/Transfer-Learning-Library` -> `thuml/Transfer-Learning-Library`; local clone head: c4aa59e.

Root orchestration files worth knowing

`README.md` -> Acts as explicit repo evidence for workflow intent, minimal run steps, and ownership boundaries. (150 lines).

`run_assignment_osc.sh` -> Chains the per-question setups and demos into one resumable OSC job plan. (540 lines).

Functions: `usage`, `timestamp`, `log`, `quote_command`, `fail`, `on_error`, `start_stage`, `finish_stage`, `run_cmd`, `require_file`, `python_version_matches`, `python_imports_available`

`osc_gpu_batch.sh` -> Turns one repo command into a readable batch script, loads CUDA, and sources the shared GPU preflight helpers. (157 lines).

Functions: `usage`, `quote_command`

`osc_gpu_interactive.sh` -> Requests an interactive allocation and drops the user into a shell that already passed the same preflight rules. (114 lines).

Functions: `usage`, `quote_command`

`osc_gpu_preflight.sh` -> Encapsulates the repository's definition of a valid GPU allocation instead of letting each question folder guess. (82 lines).

Functions: `osc_prepare_gpu_environment`, `osc_gpu_allocation_hint`, `osc_slurm_gpu_signals_present`, `osc_visible_gpu_present`, `osc_require_gpu_allocation`

Likely repo-wide scrutiny points

- Be precise about what is original repo logic versus upstream backbone code. Over-claiming authorship of TLib, Ultralytics, YOLOv12, or FasterRCNN internals would be inaccurate.
- Be ready to explain why so much code is about validation, path handling, and environment repair: on OSC, reliability is part of the assignment's engineering content.
- For the wrapper-heavy folders, defend the repo by walking the data and control flow around the backbone, not by pretending the backbone implementation itself was written here.

Not found in repo

- A repo-wide architecture diagram is not found in repo.
- Question folders for Q1 and Q2 are not found in repo; the study guide therefore covers the actual local scope: Q3 through Q7.

Question-by-question map

- Q3: 4 files, 1454 lines, 10 output artifacts currently on disk.
- Q4: 5 files, 1344 lines, 19 output artifacts currently on disk.
- Q5: 6 files, 2783 lines, 128 output artifacts currently on disk.
- Q6: 4 files, 766 lines, 61 output artifacts currently on disk.
- Q7: 4 files, 2516 lines, 4 output artifacts currently on disk.

Q3 Faster R-CNN Object Detection

Tracked scope: 4 top-level files, 1454 lines, 32 Python definitions, 1 shell functions.

What it is

- A repo-owned CLI wrapper around the trzy/FasterRCNN repository that hides two very different execution paths behind one stable interface: CUDA-only PyTorch and a TF2 fallback.
- Its main intellectual content is runtime selection, path normalization, remote-image caching, batch handling, output planning, and compatibility shims that avoid patching vendored source files.

Why this folder exists in the repo

- To make FasterRCNN runnable on OSC with a consistent Python baseline and a headless-friendly to-file mode.
- To preserve a reviewable artifact layout under question_3_faster_rcnn/outputs/ instead of leaving results only inside the upstream repo.

Architecture and data flow

- CLI args are normalized first: repo path, target interpreter, framework choice, checkpoint path, input selection, and output destinations.
- Input handling branches three ways: a URL is downloaded into inputs/downloaded/, a single local image becomes one job, and a directory becomes a recursively discovered job list.
- Framework resolution probes the target interpreter, not the host shell: PyTorch requires both importability and CUDA visibility; TF2 requires a clean NumPy + Matplotlib + TensorFlow import set.
- PyTorch batch mode pushes one JSON job list into INLINE_PYTORCH_SCRIPT, which loads the model once and loops over every image; TF2 mode launches one subprocess per image through INLINE_TF2_SCRIPT.
- The final artifact is always an annotated image in the managed output tree or a viewer window in viewer mode.

Tracked files and responsibilities

question_3_faster_rcnn/README.md -> Repo-owned documentation/evidence file. (128 lines).

question_3_faster_rcnn/demo_fasterrcnn.py -> Repo-owned wrapper/orchestration with embedded compatibility shims. (1107 lines).

question_3_faster_rcnn/download_models_fasterrcnn.sh -> Repo-owned checkpoint download helper. (55 lines).

question_3_faster_rcnn/setup_fasterrcnn_osc.sh -> Repo-owned setup/orchestration shell script. (164 lines).

Data transformations and artifact lifecycle

- Artifact counts now present on disk: outputs 10, runs 0, datasets 0, logs 0.
- Tracked annotated outputs present: 10 files under question_3_faster_rcnn/outputs/.

Sample tracked outputs: question_3_faster_rcnn/outputs/000000000139.jpg, question_3_faster_rcnn/outputs/000000000100.jpg, question_3_faster_rcnn/outputs/0000000002157.jpg, question_3_faster_rcnn/outputs/0000000002685.jpg, question_3_faster_rcnn/outputs/0000000005193.jpg, question_3_faster_rcnn/outputs/0000000005586.jpg

Original repo logic vs upstream backbone logic

- Repo-owned logic here is mostly the tracked .py/.sh wrapper layer plus any artifact shaping, environment enforcement, and summary-writing logic called out below.
- Hands off to external/FasterRCNN/pytorch/* through the embedded PyTorch script and external/FasterRCNN/tf2/* through the embedded TF2 script.
- Imports in the embedded script point directly to upstream FasterRCNN modules such as pytorch.FasterRCNN.*, tf2.FasterRCNN.*, and their visualization/model helpers.
- Downloads checkpoints expected by the upstream FasterRCNN code paths and stores them under the upstream clone root.
- Clones <https://github.com/trzy/FasterRCNN.git> into external/FasterRCNN and installs requirements from pytorch/requirements.txt and tf2/requirements.txt inside that clone.

What to explain deeply vs what to mention briefly

- Explain deeply: Explain resolve_framework(...) carefully because it is where the wrapper decides whether the run is valid at all.

- Explain deeply: Explain `resolve_input_selection(...)` and `resolve_output_paths(...)` because these are the main repo-owned transformations from user input into deterministic jobs.
- Explain deeply: Explain why the wrapper keeps compatibility fixes inline instead of patching the upstream clone in place.
- Mention briefly: Mention the low-level FasterRCNN detector math only as upstream backbone behavior unless you also reviewed `external/FasterRCNN/*` in depth.
- Mention briefly: Mention the download shell helper and default URLs briefly; they are operational, not algorithmically central.

Minimal run path

- Setup: `bash question_3_faster_rcnn/setup_fasterrcnn_osc.sh`
- Weights: `bash question_3_faster_rcnn/download_models_fasterrcnn.sh`
- CPU-capable default: `python3.9 question_3_faster_rcnn/demo_fasterrcnn.py --mode to-file`
- Explicit GPU path: `bash osc_gpu_batch.sh --account <OSC_ACCOUNT> --time 01:00:00 -- python3.9 question_3_faster_rcnn/demo_fasterrcnn.py --framework pytorch --mode to-file`

Per-file walkthrough

File: `question_3_faster_rcnn/README.md`

Classification: Repo-owned documentation/evidence file. Evidence anchor: `question_3_faster_rcnn/README.md:1-128`

Purpose: Acts as explicit repo evidence for workflow intent, minimal run steps, and ownership boundaries.

Key headings present in the tracked README: `Faster R-CNN Object Detection`, `Overview`, `Setup`, `Run`, `Inputs and Outputs`, `Environment Notes`, `Troubleshooting`

File: `question_3_faster_rcnn/demo_fasterrcnn.py`

Classification: Repo-owned wrapper/orchestration with embedded compatibility shims. Evidence anchor: `question_3_faster_rcnn/demo_fasterrcnn.py:1-1107`

Purpose: Keeps one stable CLI while switching between CUDA-only PyTorch FasterRCNN and the TF2 fallback, normalizing paths, downloads, batch handling, and output paths.

- Upstream handoff: Hands off to `external/FasterRCNN/pytorch/*` through the embedded PyTorch script and `external/FasterRCNN/tf2/*` through the embedded TF2 script.
- Upstream handoff: Imports in the embedded script point directly to upstream FasterRCNN modules such as `pytorch.FasterRCNN.*`, `tf2.FasterRCNN.*`, and their visualization/model helpers.

Top import evidence: `__future__`: `annotations`, `argparse`, `hashlib`, `json`, `os`, `shlex`, `subprocess`, `sys`

Embedded runtime scripts executed with `python -c`

`INLINE_PYTORCH_SCRIPT` lives in `question_3_faster_rcnn/demo_fasterrcnn.py:55-161`.

- Loads the FasterRCNN PyTorch model once, restores Pillow `getsize()` compatibility, iterates over a JSON job list, and calls upstream visualization on each result.
- This is important because the outer wrapper never imports upstream PyTorch code directly; it executes this script inside the target runtime with `python -c`.

Inner definition: `compat_getsize` at `question_3_faster_rcnn/demo_fasterrcnn.py:70-75`. Summary: Implements `compat_getsize` as a local helper logic helper.

Inner definition: `build_backbone` at `question_3_faster_rcnn/demo_fasterrcnn.py:84-97`. Summary: Implements `build_backbone` as a configuration or artifact construction helper.

Inner definition: `predict_job` at `question_3_faster_rcnn/demo_fasterrcnn.py:100-115`. Summary: Implements `predict_job` as a local helper logic helper.

`INLINE_TF2_SCRIPT` lives in `question_3_faster_rcnn/demo_fasterrcnn.py:166-316`.

- Builds the TF2 model with a concrete input shape before loading H5 weights, patches the text renderer for newer Pillow behavior, and then runs inference plus drawing.
- The warm-up forward pass before `load_weights(...)` is one of the most scrutiny-worthy lines because it exists purely to satisfy upstream variable naming/layout constraints.

Inner definition: `compat_draw_text` at `question_3_faster_rcnn/demo_fasterrcnn.py:204-222`. Summary: Implements `compat_draw_text` as a local helper logic helper.

Inner definition: `compat_show_detections` at `question_3_faster_rcnn/demo_fasterrcnn.py:228-249`. Summary: Implements `compat_show_detections` as a local helper logic helper.

Function-by-function walkthrough

ImageJob (class; `question_3_faster_rcnn/demo_fasterrcnn.py:321-326`)

Category: `class/type` definition. Summary: One resolved inference job for a local image path.

InputSelection (class; `question_3_faster_rcnn/demo_fasterrcnn.py:332-337`)

Category: `class/type` definition. Summary: A normalized file-or-directory selection resolved from `--image`.

is_url (function; `question_3_faster_rcnn/demo_fasterrcnn.py:343-345`)

Category: `local helper logic`. Summary: Return True when the value looks like a URL.

infer_backbone (function; `question_3_faster_rcnn/demo_fasterrcnn.py:348-363`)

Category: `local helper logic`. Summary: Infer the backbone name from the weight path.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:351-363`: Some upstream PyTorch checkpoints require the backbone name separately, so infer it from the filename when `--backbone` is omitted.

infer_default_framework (function; `question_3_faster_rcnn/demo_fasterrcnn.py:366-370`)

Category: `local helper logic`. Summary: Pick the default framework mode for this machine.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:369-370`: Resolve the practical runtime from the target environment instead of the host platform so one default command works on both OSC GPU and CPU nodes.

resolve_python (function; `question_3_faster_rcnn/demo_fasterrcnn.py:373-382`)

Category: `path/config resolution`. Summary: Resolve the Python executable to use.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:376-382`: Prefer the repository-local virtual environment when it exists.

run_python_probe (function; `question_3_faster_rcnn/demo_fasterrcnn.py:385-396`)

Category: `stage execution`. Summary: Run a short Python probe in the target interpreter.

resolve_target_python_version (function; `question_3_faster_rcnn/demo_fasterrcnn.py:399-411`)

Category: `path/config resolution`. Summary: Ask one target interpreter for its major.minor version.

Failure signals raised directly here: `RuntimeError`

ensure_supported_runtime_python (function; `question_3_faster_rcnn/demo_fasterrcnn.py:414-430`)

Category: `validation or readiness gate`. Summary: Reject unsupported target interpreters before probing frameworks.

Failure signals raised directly here: `RuntimeError`

pytorch_import_available (function; `question_3_faster_rcnn/demo_fasterrcnn.py:433-436`)

Category: `local helper logic`. Summary: Check whether PyTorch imports in the target interpreter.

pytorch_cuda_available (function; `question_3_faster_rcnn/demo_fasterrcnn.py:439-448`)

Category: `local helper logic`. Summary: Check whether PyTorch reports CUDA support.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:442-448`: The upstream PyTorch path is CUDA-only, so check that before launching a run that would otherwise fail after all path resolution is already done.

tf2_runtime_available (function; `question_3_faster_rcnn/demo_fasterrcnn.py:451-463`)

Category: `local helper logic`. Summary: Check whether the TF2 fallback runtime imports cleanly.

osc_gpu_batch_example (function; `question_3_faster_rcnn/demo_fasterrcnn.py:466-471`)

Category: `local helper logic`. Summary: Build one OSC GPU launcher example command.

resolve_framework (function; `question_3_faster_rcnn/demo_fasterrcnn.py:474-520`)

Category: `path/config resolution`. Summary: This is the real gatekeeper for Q3: it proves whether the prepared environment can legally run PyTorch, must fall back to TF2, or should fail before any upstream code launches.

Failure signals raised directly here: `RuntimeError`

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:475-520: Single contiguous control block.

resolve_weights (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:525-543)

Category: `path/config resolution`. Summary: Resolve weights.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:528-530: Leave remote weight URLs unchanged.; `question_3_faster_rcnn/demo_fasterrcnn.py`:532-535: Expand "~" and anchor relative paths to the FasterRCNN repo clone.; `question_3_faster_rcnn/demo_fasterrcnn.py`:537-543: Stop before launching inference if the checkpoint file is missing.

remote_cache_path (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:546-559)

Category: `local helper logic`. Summary: Build the cache path for a remote image.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:549: Cache remote images under a deterministic hashed filename so repeated runs reuse the same downloaded file instead of downloading it again.; `question_3_faster_rcnn/demo_fasterrcnn.py`:551-554: Pull the original filename pieces from the URL path.; `question_3_faster_rcnn/demo_fasterrcnn.py`:556-559: Add a short URL hash so different URLs with the same filename do not collide.

cache_remote_image (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:562-595)

Category: `subprocess orchestration`. Summary: Download and cache one remote image when needed.

Failure signals raised directly here: `RuntimeError`

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:566-568: In dry-run mode, only report the cache path that would be used.; `question_3_faster_rcnn/demo_fasterrcnn.py`:571: Keep downloaded examples under inputs/downloaded so they are visible in the project tree but still separate from the manually curated input images.; `question_3_faster_rcnn/demo_fasterrcnn.py`:573-575: Reuse the cached file when it already exists and is non-empty.; `question_3_faster_rcnn/demo_fasterrcnn.py`:577-587: Download the remote image with curl so this file can capture any error text.; `question_3_faster_rcnn/demo_fasterrcnn.py`:589-595: Raise one script-level error when curl fails.

resolve_local_input (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:598-632)

Category: `path/config resolution`. Summary: Resolve a local input path.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:604-605: Absolute paths need no extra search roots.; `question_3_faster_rcnn/demo_fasterrcnn.py`:608-618: Search relative to the current shell directory, then this directory, then the upstream repository.; `question_3_faster_rcnn/demo_fasterrcnn.py`:620-627: Resolve each candidate path before comparing or returning it.; `question_3_faster_rcnn/demo_fasterrcnn.py`:629-632: If nothing matched, show every path that was checked.

discover_directory_images (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:635-657)

Category: `local helper logic`. Summary: Find supported images under the input directory.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:644-649: If `--image` points at the whole inputs/ directory, skip the downloaded-cache subtree so batch runs only process the curated examples.; `question_3_faster_rcnn/demo_fasterrcnn.py`:651-657: Stop if the directory tree did not yield any supported images.

resolve_input_selection (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:660-714)

Category: `path/config resolution`. Summary: Converts the raw `--image` argument into a normalized job list so the rest of the wrapper can treat single-file, directory, and URL flows uniformly.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:663-664: Convert `--image` into one normalized list of jobs. Everything downstream can then treat single-image and batch mode uniformly.; `question_3_faster_rcnn/demo_fasterrcnn.py`:666-669: Download the URL into the local cache and build a one-job selection.; `question_3_faster_rcnn/demo_fasterrcnn.py`:671-686: Make sure the output name still has an image suffix.; `question_3_faster_rcnn/demo_fasterrcnn.py`:688-702: Expand a directory input into one `ImageJob` per discovered image.; `question_3_faster_rcnn/demo_fasterrcnn.py`:704-714: Otherwise build a one-job selection for the single local file.

resolve_single_output_path (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:717-732)

Category: `path/config resolution`. Summary: Resolve the output path for a single-image run.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py`:720-724: For single-image runs, accept either an explicit filename or a directory and fill in the actual output filename from the input image name when needed.; `question_3_faster_rcnn/demo_fasterrcnn.py`:726-727: Treat an existing directory path as an output folder.; `question_3_faster_rcnn/demo_fasterrcnn.py`:729-730: Treat a suffix-less path as a directory-like destination too.; `question_3_faster_rcnn/demo_fasterrcnn.py`:732: Otherwise treat the value as an explicit file path.

resolve_directory_output_root (function; `question_3_faster_rcnn/demo_fasterrcnn.py`:735-755)

Category: `path/config resolution`. Summary: Resolve the output root for a directory run.

Failure signals raised directly here: `ValueError`

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:738-742`: Directory runs must resolve to one output root because each input image keeps its relative name underneath that root.; `question_3_faster_rcnn/demo_fasterrcnn.py:744-748`: Reject existing files as output roots for batch mode.; `question_3_faster_rcnn/demo_fasterrcnn.py:750-755`: Reject suffix-looking paths that do not exist yet.

resolve_output_paths (function; `question_3_faster_rcnn/demo_fasterrcnn.py:758-775`)

Category: `path/config resolution`. Summary: Maps the normalized input selection into deterministic destination paths and enforces viewer-vs-file behavior.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:762-765`: Viewer mode leaves output handling to the upstream GUI/image viewer path. Viewer mode does not write file outputs managed by this script.; `question_3_faster_rcnn/demo_fasterrcnn.py:767-772`: Batch mode maps each job's relative path under one output root.; `question_3_faster_rcnn/demo_fasterrcnn.py:774-775`: Single-image mode resolves exactly one output path.

build_parser (function; `question_3_faster_rcnn/demo_fasterrcnn.py:779-849`)

Category: `configuration or artifact construction`. Summary: Build the CLI parser.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:780-849`: Single contiguous control block.

format_run_summary (function; `question_3_faster_rcnn/demo_fasterrcnn.py:852-884`)

Category: `summary/report formatting`. Summary: Build the short run summary text.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:855-856`: Print a compact summary before execution so the high-level plan is visible without reading the full subprocess command.; `question_3_faster_rcnn/demo_fasterrcnn.py:858-871`: Describe the batch run in a compact multi-line block.; `question_3_faster_rcnn/demo_fasterrcnn.py:873-884`: Otherwise describe the single-image run.

build_command (function; `question_3_faster_rcnn/demo_fasterrcnn.py:889-908`)

Category: `upstream command construction`. Summary: Build the upstream inference command.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:893-904`: The TF2 path executes the compatibility shim above because it handles model warm-up, H5 loading, and drawing in a way that works reliably on this setup.; `question_3_faster_rcnn/demo_fasterrcnn.py:906-908`: Viewer mode adds the flag that opens the rendered image window.

build_pytorch_jobs_payload (function; `question_3_faster_rcnn/demo_fasterrcnn.py:911-919`)

Category: `configuration or artifact construction`. Summary: Build the job list consumed by the inline PyTorch batch runner.

build_pytorch_batch_command (function; `question_3_faster_rcnn/demo_fasterrcnn.py:922-938`)

Category: `upstream command construction`. Summary: Packages the entire batch into one `python -c` call so the upstream PyTorch model loads once and reuses the same process across all images.

Failure signals raised directly here: `ValueError`

build_subprocess_env (function; `question_3_faster_rcnn/demo_fasterrcnn.py:941-951`)

Category: `configuration or artifact construction`. Summary: Build the environment for the selected upstream command.

print_command (function; `question_3_faster_rcnn/demo_fasterrcnn.py:954-958`)

Category: `local helper logic`. Summary: Print the command before running it.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:957-958`: Prefix batch commands with their position in the run.

print_batch_item (function; `question_3_faster_rcnn/demo_fasterrcnn.py:961-967`)

Category: `local helper logic`. Summary: Print the batch item being processed.

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:964-965`: Print the input image currently being processed.; `question_3_faster_rcnn/demo_fasterrcnn.py:967`: Print the expected output location for this image.

main (function; `question_3_faster_rcnn/demo_fasterrcnn.py:970-1091`)

Category: `entrypoint orchestration`. Summary: Binds the whole wrapper together: validate the clone, select framework, resolve weights/inputs/outputs, print the run summary, and then dispatch to the chosen upstream path.

Failure signals raised directly here: `FileNotFoundError`, `ValueError`

Line-block walk: `question_3_faster_rcnn/demo_fasterrcnn.py:976-989`: First verify that the requested repo really contains the expected upstream code.; `question_3_faster_rcnn/demo_fasterrcnn.py:993-996`: Resolve the inputs once so the execution loop can stay simple. At this point the file knows which framework, weights, inputs, and outputs will be used before it launches any upstream inference code.; `question_3_faster_rcnn/demo_fasterrcnn.py:998-1003`: Reject viewer mode for directory batches because the upstream tools show one image at a time.; `question_3_faster_rcnn/demo_fasterrcnn.py:1006-1007`: Output path resolution mirrors the normalized input selection so each job has one concrete destination in file-writing mode.; `question_3_faster_rcnn/demo_fasterrcnn.py:1012-1047`: Print one high-level summary before any subprocesses start.; `question_3_faster_rcnn/demo_fasterrcnn.py:1050`: Each job becomes one upstream subprocess call so single-image and batch modes both reuse the same execution path.; `question_3_faster_rcnn/demo_fasterrcnn.py:1053-1066`: Each iteration turns one normalized

job description into the exact upstream command line needed for TF2 or PyTorch inference.;
question_3_faster_rcnn/demo_fasterrcnn.py:1068-1072: In quiet batch mode, print just the current image and destination.

File: question_3_faster_rcnn/download_models_fasterrcnn.sh

Classification: Repo-owned checkpoint download helper. Evidence anchor:
question_3_faster_rcnn/download_models_fasterrcnn.sh:1-55

Purpose: Downloads the model weights that the wrapper expects to find under the FasterRCNN clone.

- Upstream handoff: Downloads checkpoints expected by the upstream FasterRCNN code paths and stores them under the upstream clone root.

Top-level shell stages

question_3_faster_rcnn/download_models_fasterrcnn.sh:2-3: `!/usr/bin/env bash`
question_3_faster_rcnn/download_models_fasterrcnn.sh:5-4: Download the tracked FasterRCNN checkpoints into the local upstream clone.
question_3_faster_rcnn/download_models_fasterrcnn.sh:6-5: This script intentionally stays small because the repo-root wrappers already
question_3_faster_rcnn/download_models_fasterrcnn.sh:7-12: handle environment selection and only need the weight files to exist.
question_3_faster_rcnn/download_models_fasterrcnn.sh:14-16: Resolve relative repo paths before the rest of the script uses them.
question_3_faster_rcnn/download_models_fasterrcnn.sh:18-23: Stop early if setup has not created the upstream clone yet.
question_3_faster_rcnn/download_models_fasterrcnn.sh:25: Try the available download tools in order.
question_3_faster_rcnn/download_models_fasterrcnn.sh:27-43: Prefer wget when present, but keep curl as a compatible fallback.
question_3_faster_rcnn/download_models_fasterrcnn.sh:45-46: Change into the upstream repository before writing the checkpoint files.
question_3_faster_rcnn/download_models_fasterrcnn.sh:48-47: Download the supported base and detector weights into the repository root.
question_3_faster_rcnn/download_models_fasterrcnn.sh:49-55: The wrappers infer the correct backend from these filenames later.

Shell functions

download_file at question_3_faster_rcnn/download_models_fasterrcnn.sh:25-55. Category: download or remote fetch. Summary:
Implements download_file as a download or remote fetch helper.

Representative commands and side effects

```
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
REPO_DIR="${1:-${REPO_ROOT}/external/FasterRCNN}"
if [[ "${REPO_DIR}" != /* ]]; then
REPO_DIR="${PWD}/${REPO_DIR}"
if [[ ! -d "${REPO_DIR}" ]]; then
echo "Error: FasterRCNN repo not found at ${REPO_DIR}"
```

File: question_3_faster_rcnn/setup_fasterrcnn_osc.sh

Classification: Repo-owned setup/orchestration shell script. Evidence anchor:
question_3_faster_rcnn/setup_fasterrcnn_osc.sh:1-164

Purpose: Prepares the local FasterRCNN clone and one OSC-safe environment that can support both the PyTorch and TF2 code paths.

- Upstream handoff: Clones <https://github.com/trzy/FasterRCNN.git> into external/FasterRCNN and installs requirements from pytorch/requirements.txt and tf2/requirements.txt inside that clone.

Top-level shell stages

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:2-3: `!/usr/bin/env bash`
question_3_faster_rcnn/setup_fasterrcnn_osc.sh:5-4: Prepare the repo-local FasterRCNN clone and virtual environment used by the
question_3_faster_rcnn/setup_fasterrcnn_osc.sh:6-5: tracked Q3 wrapper. The setup keeps both the CUDA-backed PyTorch path and the
question_3_faster_rcnn/setup_fasterrcnn_osc.sh:7-18: optional TF2 CPU fallback under one reproducible OSC baseline.
question_3_faster_rcnn/setup_fasterrcnn_osc.sh:20-19: Install the TF2 fallback by default so one OSC setup command prepares both the
question_3_faster_rcnn/setup_fasterrcnn_osc.sh:21-24: CUDA PyTorch path and the CPU-capable fallback runtime.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:26-34: Prefer Python 3.9 when it is available on OSC.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:36-38: Resolve relative clone targets before the rest of the script uses the path.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:40-45: Stop immediately if the requested interpreter is unavailable.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:47-46: Clone or reuse the upstream repository before rebuilding the environment.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:48-49: Create the parent directory before cloning into it.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:51-59: Reuse the existing clone when it is already present.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:61-70: Capture the selected interpreter version before reusing or rebuilding the environment.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:72-79: Refuse to reuse an environment built with a different Python minor version.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:81-83: Rebuild the virtual environment from the selected interpreter.

question_3_faster_rcnn/setup_fasterrcnn_osc.sh:85-86: Activate the environment before installing packages into it.

Representative commands and side effects

```
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"

DEFAULT_REPO_DIR="${REPO_ROOT}/external/FasterRCNN"

REPO_URL="${REPO_URL:-https://github.com/trzy/FasterRCNN.git}"

REPO_DIR="{1:-${DEFAULT_REPO_DIR}}"

OS_NAME="$(uname -s)"

PYTHON_BIN="{PYTHON_BIN:-}"
```

Not found in repo

- A quantitative evaluation report for Q3 is not found in repo; the tracked outputs are annotated images, not benchmark tables.
- A formal per-line diff against upstream FasterRCNN is not found in repo; the wrapper shows the handoff points, but not a provenance ledger for every borrowed idea.

Q4 YOLO11 and YOLOv12 Object Detection

Tracked scope: 5 top-level files, 1344 lines, 38 Python definitions, 0 shell functions.

What it is

- A paired set of wrappers that make YOLO11 and YOLOv12 look operationally identical from the assignment side even though they come from different external clones.
- The repo-owned logic is mostly about standardizing runtime checks, source normalization, output copying, and OSC guardrails across both models.

Why this folder exists in the repo

- To compare two YOLO-family backbones under one shared repo style.
- To keep outputs under dedicated per-model folders while still reusing the same wrapper pattern.

Architecture and data flow

- Both wrappers resolve the upstream repo and target interpreter, enforce the Python 3.9.18 baseline, and reject GPU requests when CUDA is unavailable.
- The source may be a local file, a local directory, or a URL. The wrapper predicts exactly what filenames the upstream save path should emit before it runs the model.
- The inline script is intentionally tiny: instantiate YOLO, build `predict_kwargs`, and call `predict` inside the target environment.
- After the upstream run, the wrapper copies curated artifacts from `runs/detect/<name>/` into `outputs/yolo11/` or `outputs/yolov12/` and surfaces readable status messages.

Tracked files and responsibilities

`question_4_yolo11_yolov12/README.md` -> Repo-owned documentation/evidence file. (120 lines).

`question_4_yolo11_yolov12/demo_yolo11.py` -> Repo-owned wrapper/orchestration for Ultralytics YOLO11. (492 lines).

`question_4_yolo11_yolov12/demo_yolov12.py` -> Repo-owned wrapper/orchestration for the YOLOv12 clone. (509 lines).

`question_4_yolo11_yolov12/setup_yolo11_osc.sh` -> Repo-owned Ultralytics environment setup script. (94 lines).

`question_4_yolo11_yolov12/setup_yolov12_osc.sh` -> Repo-owned YOLOv12 environment setup script with pinned dependencies. (129 lines).

Data transformations and artifact lifecycle

- Artifact counts now present on disk: outputs 19, runs 13, datasets 0, logs 0.
- Tracked curated outputs present: 9 YOLO11 images and 8 YOLOv12 images.

Sample tracked outputs: `question_4_yolo11_yolov12/outputs/yolo11`, `question_4_yolo11_yolov12/outputs/yolo11/.gitkeep`, `question_4_yolo11_yolov12/outputs/yolo11/000000000139.jpg`, `question_4_yolo11_yolov12/outputs/yolo11/0000000001000.jpg`, `question_4_yolo11_yolov12/outputs/yolo11/0000000002157.jpg`, `question_4_yolo11_yolov12/outputs/yolo11/0000000002685.jpg`

Sample tracked runs: `question_4_yolo11_yolov12/runs/detect`, `question_4_yolo11_yolov12/runs/detect/yolo11_demo`, `question_4_yolo11_yolov12/runs/detect/yolo11_demo/000000000139.jpg`, `question_4_yolo11_yolov12/runs/detect/yolo11_demo/0000000001000.jpg`, `question_4_yolo11_yolov12/runs/detect/yolo11_demo/0000000002157.jpg`, `question_4_yolo11_yolov12/runs/detect/yolo11_demo/0000000002685.jpg`

Original repo logic vs upstream backbone logic

- Repo-owned logic here is mostly the tracked `.py/.sh` wrapper layer plus any artifact shaping, environment enforcement, and summary-writing logic called out below.
- The inline runtime script imports `ultralytics.YOLO` from `external/ultralytics` and delegates prediction to `model.predict(...)`.
- The inline runtime script imports `ultralytics.YOLO` from `external/yolov12` and delegates prediction to `model.predict(...)`.
- Clones the official Ultralytics repo and installs it editable so imports resolve to the local source tree.
- Clones the YOLOv12 repo and installs its local source tree after pinning a repo-owned dependency set.

What to explain deeply vs what to mention briefly

- Explain deeply: Explain the mirrored design between `demo_yolo11.py` and `demo_yolov12.py` because that is clear evidence of repo-authored interface shaping rather than copied model code.
- Explain deeply: Explain `expected_output_names(...)` plus `copy_saved_images(...)`, since those functions are what make the wrapper deterministic and reviewable.
- Explain deeply: Explain why YOLOv12 setup pins a narrower dependency set than the upstream repo: the repo is asserting a reproducible OSC path.
- Mention briefly: Mention the actual YOLO backbone implementation only as upstream logic unless you inspected the external clones deeply.
- Mention briefly: Mention the ANSI log cleaning and warning rewriting briefly; it matters operationally but is not the folder's main idea.

Minimal run path

- YOLO11 setup: `bash question_4_yolo11_yolov12/setup_yolo11_osc.sh`
- YOLOv12 setup: `bash question_4_yolo11_yolov12/setup_yolov12_osc.sh`
- YOLO11 CPU run: `python3.9 question_4_yolo11_yolov12/demo_yolo11.py --device cpu`
- YOLOv12 CPU run: `python3.9 question_4_yolo11_yolov12/demo_yolov12.py --device cpu`

Per-file walkthrough

File: `question_4_yolo11_yolov12/README.md`

Classification: Repo-owned documentation/evidence file. Evidence anchor: `question_4_yolo11_yolov12/README.md:1-120`

Purpose: Acts as explicit repo evidence for workflow intent, minimal run steps, and ownership boundaries.

Key headings present in the tracked README: YOLO11 and YOLOv12 Object Detection, Overview, Setup, Run, Inputs and Outputs, Environment Notes

File: `question_4_yolo11_yolov12/demo_yolo11.py`

Classification: Repo-owned wrapper/orchestration for Ultralytics YOLO11. Evidence anchor: `question_4_yolo11_yolov12/demo_yolo11.py:1-492`

Purpose: Resolves paths, validates the runtime, runs Ultralytics YOLO11 through an inline script, then copies curated artifacts out of the upstream run folder.

- Upstream handoff: The inline runtime script imports `ultralytics.YOLO` from `external/ultralytics` and delegates prediction to `model.predict(...)`.

Top import evidence: `__future__::annotations`, `argparse`, `re`, `shlex`, `shutil`, `subprocess`, `sys`, `pathlib::Path`

Embedded runtime scripts executed with `python -c`

INLINE_SCRIPT lives in `question_4_yolo11_yolov12/demo_yolo11.py:33-67`.

- This embedded script is the minimal upstream handoff: parse args, instantiate YOLO, build `predict_kwargs`, optionally inject device, and call `model.predict(...)` once.

Function-by-function walkthrough

resolve_python (function; `question_4_yolo11_yolov12/demo_yolo11.py:74-82`)

Category: path/config resolution. Summary: Resolve the Python executable used for inference.

Line-block walk: `question_4_yolo11_yolov12/demo_yolo11.py:77-82`: Prefer the repository-local virtual environment when it exists.

resolve_target_python_version (function; `question_4_yolo11_yolov12/demo_yolo11.py:85-103`)

Category: path/config resolution. Summary: Ask one target interpreter for its major.minor version.

Failure signals raised directly here: `RuntimeError`

Line-block walk: `question_4_yolo11_yolov12/demo_yolo11.py:86-103`: Single contiguous control block.

ensure_supported_runtime_python (function; `question_4_yolo11_yolov12/demo_yolo11.py:106-115`)

Category: validation or readiness gate. Summary: Reject unsupported target interpreters before launching YOLO.

Failure signals raised directly here: `RuntimeError`

device_requests_gpu (function; question_4_yolo11_yolov12/demo_yolo11.py:118-121)

Category: local helper logic. Summary: Return True when the selected device expects CUDA.

runtime_cuda_available (function; question_4_yolo11_yolov12/demo_yolo11.py:124-136)

Category: subprocess orchestration. Summary: Return True when the target runtime reports CUDA availability.

ensure_gpu_device_ready (function; question_4_yolo11_yolov12/demo_yolo11.py:139-152)

Category: validation or readiness gate. Summary: Fail fast when CUDA was requested without a GPU-ready runtime.

Failure signals raised directly here: RuntimeError

resolve_project (function; question_4_yolo11_yolov12/demo_yolo11.py:155-163)

Category: path/config resolution. Summary: Resolve the Ultralytics run-root directory.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:159-160: YOLO writes raw run artifacts under the project directory first. Expand "~" before anchoring relative paths.; question_4_yolo11_yolov12/demo_yolo11.py:162-163: Anchor relative project paths to this directory.

resolve_output_dir (function; question_4_yolo11_yolov12/demo_yolo11.py:166-172)

Category: path/config resolution. Summary: Resolve the output directory path.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:171-172: Anchor relative output directories to this directory.

resolve_output_path (function; question_4_yolo11_yolov12/demo_yolo11.py:175-181)

Category: path/config resolution. Summary: Resolve one explicit output file path.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:180-181: Anchor relative output files to this directory.

describe_source (function; question_4_yolo11_yolov12/demo_yolo11.py:184-193)

Category: local helper logic. Summary: Build a short description of the resolved source selection.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:187-193: Keep the printed run summary readable for URLs, files, and directories.

clean_log_lines (function; question_4_yolo11_yolov12/demo_yolo11.py:196-201)

Category: local helper logic. Summary: Normalize and split the log output.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:199: Replace carriage returns before splitting the captured logs into lines.; question_4_yolo11_yolov12/demo_yolo11.py:201: Drop empty lines and surrounding whitespace.

runtime_messages (function; question_4_yolo11_yolov12/demo_yolo11.py:204-222)

Category: local helper logic. Summary: Keep the short runtime messages worth printing.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:207-208: Keep only the short runtime notes that explain how the run behaved.; question_4_yolo11_yolov12/demo_yolo11.py:210-211: Rewrite the FlashAttention warning into a shorter status message.; question_4_yolo11_yolov12/demo_yolo11.py:213-215: Skip the raw save-path line because this script prints curated paths later.; question_4_yolo11_yolov12/demo_yolo11.py:217-218: Drop duplicate messages before printing them.; question_4_yolo11_yolov12/demo_yolo11.py:220-222: Preserve the first copy of each message only.

is_url (function; question_4_yolo11_yolov12/demo_yolo11.py:227-230)

Category: local helper logic. Summary: Return True when the value looks like a URL.

resolve_source (function; question_4_yolo11_yolov12/demo_yolo11.py:233-251)

Category: path/config resolution. Summary: Resolve the input source into a URL or one absolute path.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:237-239: Normalize the source value into either a raw URL string or one absolute local path. Return URLs unchanged and mark them as non-path inputs.; question_4_yolo11_yolov12/demo_yolo11.py:241-242: Expand "~" before anchoring a local source path.; question_4_yolo11_yolov12/demo_yolo11.py:244-246: Anchor relative source paths to this directory.; question_4_yolo11_yolov12/demo_yolo11.py:248-251: Stop before launching inference if the local path is missing.

list_supported_images (function; question_4_yolo11_yolov12/demo_yolo11.py:254-265)

Category: local helper logic. Summary: List the supported images under one directory.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:259-265: Batch mode uses only top-level files so the copied outputs keep one predictable filename per input image. Return one sorted list of top-level supported image files.

expected_output_names (function; question_4_yolo11_yolov12/demo_yolo11.py:268-296)

Category: local helper logic. Summary: Predicts the filenames YOLO should emit before the run, which is what lets the wrapper detect missing artifacts cleanly afterward.

Failure signals raised directly here: FileNotFoundError, ValueError

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:272-275: Predict the filenames YOLO will save so the later copy step knows exactly which artifacts to collect.; question_4_yolo11_yolov12/demo_yolo11.py:277-279: Use the URL filename directly for the saved image name.; question_4_yolo11_yolov12/demo_yolo11.py:281-288: In directory mode, expect one saved image per discovered input file.; question_4_yolo11_yolov12/demo_yolo11.py:290-296: For a single local file, verify the suffix before using its name.

copy_saved_images (function; question_4_yolo11_yolov12/demo_yolo11.py:299-345)

Category: artifact copying or dataset materialization. Summary: Moves the raw upstream save-dir outputs into the curated assignment output directory while validating that every expected image actually exists.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:303-307: Ultralytics saves raw outputs into the run directory first, then this helper copies the final annotated images into the local output location.; question_4_yolo11_yolov12/demo_yolo11.py:309: Single-image mode can write directly to one requested destination path.; question_4_yolo11_yolov12/demo_yolo11.py:311-315: Stop if the expected saved image does not exist.; question_4_yolo11_yolov12/demo_yolo11.py:317: Create the destination folder before copying the image.; question_4_yolo11_yolov12/demo_yolo11.py:319-321: Copy the one saved image to the requested output path.; question_4_yolo11_yolov12/demo_yolo11.py:323-327: Directory mode writes one copied file per expected saved image.; question_4_yolo11_yolov12/demo_yolo11.py:329: Directory mode preserves the original filenames so outputs line up with inputs.; question_4_yolo11_yolov12/demo_yolo11.py:331-334: Record any expected file that YOLO did not save.

build_parser (function; question_4_yolo11_yolov12/demo_yolo11.py:349-389)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:350-389: Single contiguous control block.

main (function; question_4_yolo11_yolov12/demo_yolo11.py:392-477)

Category: entrypoint orchestration. Summary: Shows the full repo-owned contract for YOLO11: repo validation, interpreter checks, source normalization, command construction, upstream run, and curated artifact copy-out.

Failure signals raised directly here: FileNotFoundError, ValueError

Line-block walk: question_4_yolo11_yolov12/demo_yolo11.py:398-404: Validate that the requested upstream checkout is really the Ultralytics repo.; question_4_yolo11_yolov12/demo_yolo11.py:406-413: Resolve every path before launching YOLO so the artifact copy step stays deterministic.; question_4_yolo11_yolov12/demo_yolo11.py:415-417: Reject a single-file output path when the source expands to multiple images.; question_4_yolo11_yolov12/demo_yolo11.py:419-420: Match the post-run copy step to whatever filenames YOLO is expected to emit.; question_4_yolo11_yolov12/demo_yolo11.py:423-443: Launch one inline Python snippet inside the repository environment so this file controls imports and arguments without depending on a shell command.; question_4_yolo11_yolov12/demo_yolo11.py:445-447: Dry-run mode prints the exact command and stops.; question_4_yolo11_yolov12/demo_yolo11.py:449-452: Run inference once, then copy the saved images into the local outputs directory.; question_4_yolo11_yolov12/demo_yolo11.py:454-455: Capture stdout and stderr so the useful run messages can be replayed on success or failure.

File: question_4_yolo11_yolov12/demo_yolov12.py

Classification: Repo-owned wrapper/orchestration for the YOLOv12 clone. Evidence anchor: question_4_yolo11_yolov12/demo_yolov12.py:1-509

Purpose: Mirrors the YOLO11 wrapper shape but points at the YOLOv12 clone and its dedicated runtime.

- Upstream handoff: The inline runtime script imports `ultralytics.YOLO` from `external/yolov12` and delegates prediction to `model.predict(...)`.

Top import evidence: `__future__`: annotations, argparse, re, shlex, shutil, subprocess, sys, pathlib: Path

Embedded runtime scripts executed with python -c

INLINE_SCRIPT lives in question_4_yolo11_yolov12/demo_yolov12.py:33-67.

- Same control shape as the YOLO11 inline script, but executed inside the YOLOv12 environment and pointed at its checkpoint names and clone path.

Function-by-function walkthrough

resolve_python (function; question_4_yolo11_yolov12/demo_yolov12.py:74-86)

Category: path/config resolution. Summary: Unlike YOLO11, YOLOv12 refuses to proceed if its prepared .venv is missing; this is a deliberate stricter contract because the runtime is more tightly pinned.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:77-86: Respect an explicit interpreter override first.

resolve_target_python_version (function; question_4_yolo11_yolov12/demo_yolov12.py:89-107)

Category: path/config resolution. Summary: Ask one target interpreter for its major.minor version.

Failure signals raised directly here: RuntimeError

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:90-107: Single contiguous control block.

ensure_supported_runtime_python (function; question_4_yolo11_yolov12/demo_yolov12.py:110-119)

Category: validation or readiness gate. Summary: Reject unsupported target interpreters before launching YOLO.

Failure signals raised directly here: RuntimeError

device_requests_gpu (function; question_4_yolo11_yolov12/demo_yolov12.py:122-125)

Category: local helper logic. Summary: Return True when the selected device expects CUDA.

runtime_cuda_available (function; question_4_yolo11_yolov12/demo_yolov12.py:128-140)

Category: subprocess orchestration. Summary: Return True when the target runtime reports CUDA availability.

ensure_gpu_device_ready (function; question_4_yolo11_yolov12/demo_yolov12.py:143-156)

Category: validation or readiness gate. Summary: Fail fast when CUDA was requested without a GPU-ready runtime.

Failure signals raised directly here: RuntimeError

resolve_project (function; question_4_yolo11_yolov12/demo_yolov12.py:159-167)

Category: path/config resolution. Summary: Resolve the Ultralytics run-root directory.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:163-164: YOLO writes raw run artifacts under the project directory first. Expand "~" before anchoring relative paths.; question_4_yolo11_yolov12/demo_yolov12.py:166-167: Anchor relative project paths to this directory.

resolve_output_dir (function; question_4_yolo11_yolov12/demo_yolov12.py:170-176)

Category: path/config resolution. Summary: Resolve the output directory path.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:175-176: Anchor relative output directories to this directory.

resolve_output_path (function; question_4_yolo11_yolov12/demo_yolov12.py:179-185)

Category: path/config resolution. Summary: Resolve one explicit output file path.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:184-185: Anchor relative output files to this directory.

describe_source (function; question_4_yolo11_yolov12/demo_yolov12.py:188-197)

Category: local helper logic. Summary: Build a short description of the resolved source selection.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:191-197: Keep the printed run summary readable for URLs, files, and directories.

clean_log_lines (function; question_4_yolo11_yolov12/demo_yolov12.py:200-205)

Category: local helper logic. Summary: Normalize and split the log output.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:203: Replace carriage returns before splitting the captured logs into lines.; question_4_yolo11_yolov12/demo_yolov12.py:205: Drop empty lines and surrounding whitespace.

runtime_messages (function; question_4_yolo11_yolov12/demo_yolov12.py:208-226)

Category: local helper logic. Summary: Keep the short runtime messages worth printing.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:211-212: Keep only the short runtime notes that explain how the run behaved.; question_4_yolo11_yolov12/demo_yolov12.py:214-215: Rewrite the FlashAttention warning into a shorter status message.; question_4_yolo11_yolov12/demo_yolov12.py:217-219: Skip the raw save-path line because this script prints curated paths later.; question_4_yolo11_yolov12/demo_yolov12.py:221-222: Drop duplicate messages before printing them.; question_4_yolo11_yolov12/demo_yolov12.py:224-226: Preserve the first copy of each message only.

is_url (function; question_4_yolo11_yolov12/demo_yolov12.py:231-234)

Category: local helper logic. Summary: Return True when the value looks like a URL.

resolve_source (function; question_4_yolo11_yolov12/demo_yolov12.py:237-255)

Category: path/config resolution. Summary: Resolve the input source into a URL or one absolute path.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:241-243: Normalize the source value into either a raw URL string or one absolute local path. Return URLs unchanged and mark them as non-path inputs.; question_4_yolo11_yolov12/demo_yolov12.py:245-246: Expand "~" before anchoring a local source path.; question_4_yolo11_yolov12/demo_yolov12.py:248-250: Anchor relative source paths to this directory.; question_4_yolo11_yolov12/demo_yolov12.py:252-255: Stop before launching inference if the local path is missing.

list_supported_images (function; question_4_yolo11_yolov12/demo_yolov12.py:258-269)

Category: local helper logic. Summary: List the supported images under one directory.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:263-269: Batch mode uses only top-level files so the copied outputs keep one predictable filename per input image. Return one sorted list of top-level supported image files.

expected_output_names (function; question_4_yolo11_yolov12/demo_yolov12.py:272-300)

Category: local helper logic. Summary: Predict which annotated filenames Ultralytics will save.

Failure signals raised directly here: FileNotFoundError, ValueError

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:276-279: Predict the filenames YOLO will save so the later copy step knows exactly which artifacts to collect.; question_4_yolo11_yolov12/demo_yolov12.py:281-283: Use the URL filename directly for the saved image name.; question_4_yolo11_yolov12/demo_yolov12.py:285-292: In directory mode, expect one saved image per discovered input file.; question_4_yolo11_yolov12/demo_yolov12.py:294-300: For a single local file, verify the suffix before using its name.

copy_saved_images (function; question_4_yolo11_yolov12/demo_yolov12.py:303-349)

Category: artifact copying or dataset materialization. Summary: Copy saved images into the final output location.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:307-311: Ultralytics saves raw outputs into the run directory first, then this helper copies the final annotated images into the local output location.; question_4_yolo11_yolov12/demo_yolov12.py:313: Single-image mode can write directly to one requested destination path.; question_4_yolo11_yolov12/demo_yolov12.py:315-319: Stop if the expected saved image does not exist.; question_4_yolo11_yolov12/demo_yolov12.py:321: Create the destination folder before copying the image.; question_4_yolo11_yolov12/demo_yolov12.py:323-325: Copy the one saved image to the requested output path.; question_4_yolo11_yolov12/demo_yolov12.py:327-331: Directory mode writes one copied file per expected saved image.; question_4_yolo11_yolov12/demo_yolov12.py:333: Directory mode preserves the original filenames so outputs line up with inputs.; question_4_yolo11_yolov12/demo_yolov12.py:335-338: Record any expected file that YOLO did not save.

build_parser (function; question_4_yolo11_yolov12/demo_yolov12.py:353-392)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:354-392: Single contiguous control block.

main (function; question_4_yolo11_yolov12/demo_yolov12.py:395-494)

Category: entrypoint orchestration. Summary: Mirrors the YOLO11 wrapper shape while surfacing stronger repo-dir error messages that point the user back to the correct setup command.

Failure signals raised directly here: FileNotFoundError, ValueError

Line-block walk: question_4_yolo11_yolov12/demo_yolov12.py:402-421: Validate that the requested upstream checkout is really the YOLOv12 repo clone.; question_4_yolo11_yolov12/demo_yolov12.py:423-430: Resolve every path before launching YOLO so the artifact copy step stays deterministic.; question_4_yolo11_yolov12/demo_yolov12.py:432-434: Reject a single-file output path when the source expands to multiple images.; question_4_yolo11_yolov12/demo_yolov12.py:436-437: Match the post-run copy step to whatever filenames YOLO is expected to emit.; question_4_yolo11_yolov12/demo_yolov12.py:440-460: Launch one inline Python snippet inside the repository environment so this file controls imports and arguments without depending on a shell command.; question_4_yolo11_yolov12/demo_yolov12.py:462-464: Dry-run mode prints the exact command and stops.; question_4_yolo11_yolov12/demo_yolov12.py:466-469: Run inference once, then copy the saved images into the local outputs directory.; question_4_yolo11_yolov12/demo_yolov12.py:471-472: Capture stdout and stderr so the useful run messages can be replayed on success or failure.

File: question_4_yolo11_yolov12/setup_yolo11_osc.sh

Classification: Repo-owned Ultralytics environment setup script. Evidence anchor: question_4_yolo11_yolov12/setup_yolo11_osc.sh:1-94

Purpose: Clones/reuses Ultralytics, rebuilds the environment, and installs the project in editable mode.

- Upstream handoff: Clones the official Ultralytics repo and installs it editable so imports resolve to the local source tree.

Top-level shell stages

question_4_yolo11_yolov12/setup_yolo11_osc.sh:2-3: !usr/bin/env bash

question_4_yolo11_yolov12/setup_yolo11_osc.sh:5-4: Prepare the repo-local Ultralytics clone used by the tracked YOLO11 wrapper.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:6-5: The script keeps the OSC interpreter baseline explicit so later wrapper

question_4_yolo11_yolov12/setup_yolo11_osc.sh:7-25: diagnostics can assume one known runtime shape.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:27-29: Resolve relative clone targets before the rest of the script uses the path.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:31-30: Clone or reuse the upstream repository before rebuilding the environment.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:32-33: Create the parent directory before cloning into it.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:35-39: Stop immediately if the requested interpreter is unavailable.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:41-49: Reuse the existing clone when it is already present.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:51-60: Capture the selected interpreter version before reusing or rebuilding the environment.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:62-69: Refuse to reuse an environment built with a different Python minor version.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:71-73: Rebuild the repository-local virtual environment used by the Python entrypoint.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:75-76: Activate the environment before installing packages into it.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:78-79: Update the packaging tools before installing the project itself.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:81-82: Install the clone in editable mode so imports resolve to this checkout.

question_4_yolo11_yolov12/setup_yolo11_osc.sh:84-94: Print the reference commands for the prepared runtime.

Representative commands and side effects

```
set -euo pipefail

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"

DEFAULT_REPO_DIR="${REPO_ROOT}/external/ultralytics"

REPO_URL="${REPO_URL:-https://github.com/ultralytics/ultralytics.git}"

REPO_DIR="{1:-${DEFAULT_REPO_DIR}}"

PYTHON_BIN="{PYTHON_BIN:-}"

SUPPORTED_PYTHON_VERSION="{SUPPORTED_PYTHON_VERSION:-3.9.18}"
```

File: question_4_yolo11_yolov12/setup_yolov12_osc.sh

Classification: Repo-owned YOLOv12 environment setup script with pinned dependencies. Evidence anchor: question_4_yolo11_yolov12/setup_yolov12_osc.sh:1-129

Purpose: Clones/reuses YOLOv12, rebuilds its environment, and pins a narrow dependency set for the supported OSC baseline.

- Upstream handoff: Clones the YOLOv12 repo and installs its local source tree after pinning a repo-owned dependency set.

Top-level shell stages

question_4_yolo11_yolov12/setup_yolov12_osc.sh:2-3: !usr/bin/env bash

question_4_yolo11_yolov12/setup_yolov12_osc.sh:5-4: Prepare the repo-local YOLOv12 clone used by the tracked wrapper. This setup

question_4_yolo11_yolov12/setup_yolov12_osc.sh:6-5: intentionally pins a narrower dependency set than the upstream project so the

question_4_yolo11_yolov12/setup_yolov12_osc.sh:7-19: OSC path stays reproducible on the shared Python 3.9.18 baseline.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:21-38: Pin only the packages required for the local YOLOv12 workflow.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:40-42: Resolve relative clone targets before the rest of the script uses the path.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:44-52: Prefer Python 3.9 when it is available, but allow a python3 fallback on OSC.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:54-53: Clone or reuse the upstream repository before rebuilding the environment.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:55-56: Create the parent directory before cloning into it.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:58-63: Stop immediately if the requested interpreter is unavailable.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:65-73: Reuse the existing clone when it is already present.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:75-84: Capture the selected interpreter version before reusing or rebuilding the environment.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:86-93: Refuse to reuse an environment built with a different Python minor version.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:95-97: Rebuild the repository-local virtual environment from the selected interpreter.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:99-100: Activate the environment before installing packages into it.

question_4_yolo11_yolov12/setup_yolov12_osc.sh:102-103: Update the packaging tools before installing project dependencies.

Representative commands and side effects

```
set -euo pipefail
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
DEFAULT_REPO_DIR="${REPO_ROOT}/external/yolov12"
REPO_URL="${REPO_URL:-https://github.com/sunsmarterjie/yolov12.git}"
REPO_DIR="${1:-${DEFAULT_REPO_DIR}}"
PYTHON_BIN="${PYTHON_BIN:-}"
INSTALL_FLASH_ATTN="${INSTALL_FLASH_ATTN:-0}"
```

Not found in repo

- A benchmark/evaluation summary comparing YOLO11 and YOLOv12 accuracy is not found in repo; tracked outputs are annotated images, not metrics tables.
- A design note explaining why `yolo11n.pt` and `yolov12n.pt` were chosen as defaults is not found in repo.

Q5 YOLO11 Semantic Segmentation on iSAID

Tracked scope: 6 top-level files, 2783 lines, 76 Python definitions, 0 shell functions.

What it is

- The most complete data-engineering pipeline in the repo: dataset discovery/download, raw-layout normalization, annotation rewriting, label conversion, training, inference, semantic fusion, and summary export.
- The architectural spine is `q5_seg_common.py`, which means the deepest oral defense should treat that file as the shared substrate behind every other Q5 entrypoint.

Why this folder exists in the repo

- To adapt the general-purpose Ultralytics segmentation workflow to one assignment-specific dataset layout and artifact contract.
- To keep the iSAID-to-YOLO conversion logic explicit and reviewable instead of burying it in ad hoc shell commands.

Architecture and data flow

- Setup ensures the Ultralytics runtime is importable and optionally auto-bootstraps the dataset and pretrained checkpoint.
- Bootstrap resolves repo-local paths, optionally downloads DOTA plus iSAID Google Drive folders, normalizes annotation JSONs to the repo's canonical 15-class scheme, converts COCO to YOLO segmentation labels, and writes `isaidd_seg.yaml`.
- Training re-execs into the repo environment, verifies CUDA when requested, subclasses the Ultralytics segmentation trainer for OSC-safe validation behavior, and copies the winning checkpoint to a stable alias path under `models/isaidd_seg/best.pt`.
- Inference loads one checkpoint, resolves any class filter, predicts one image at a time, converts instance masks into a fused semantic class map, writes overlay/class-id/mask artifacts, and serializes JSON/CSV summaries.
- The most important repo-owned data manipulations happen before and after the Ultralytics call: JSON normalization before training and semantic fusion/reporting after inference.

Tracked files and responsibilities

`question_5_semantic_segmentation/README.md` -> Repo-owned documentation/evidence file. (234 lines).

`question_5_semantic_segmentation/bootstrap_isaidd_seg.py` -> Repo-owned dataset bootstrap and normalization wrapper. (293 lines).

`question_5_semantic_segmentation/demo_yolo_segmentation.py` -> Repo-owned inference/post-processing/reporting wrapper. (777 lines).

`question_5_semantic_segmentation/q5_seg_common.py` -> Repo-owned shared implementation backbone for Q5 bootstrap, train, and demo flows. (927 lines).

`question_5_semantic_segmentation/setup_yolo11_osc.sh` -> Repo-owned environment bootstrap and optional dataset auto-bootstrap shell script. (216 lines).

`question_5_semantic_segmentation/train_isaidd_seg.py` -> Repo-owned training wrapper around Ultralytics segmentation. (336 lines).

Data transformations and artifact lifecycle

- Artifact counts now present on disk: outputs 128, runs 36, datasets 164812, logs 0.
- Existing batch evidence at `question_5_semantic_segmentation/outputs/satellite_results/index.csv`: `{'error': 8}`. The tracked CSV currently records error rows rather than silent failure.
- Representative tracked failure message: `Invalid CUDA 'device=0' requested. Use 'device=cpu' or pass valid CUDA device(s) if available, i.e. 'device=0' or 'device=0,1,2,3' for Multi-GPU.`
`torch.cuda.is_available(): False t...`
- Per-image summary artifacts present: 29 JSON files under `outputs/satellite_results/`.

Sample tracked outputs: `question_5_semantic_segmentation/outputs/.DS_Store`,
`question_5_semantic_segmentation/outputs/satellite_results`,
`question_5_semantic_segmentation/outputs/satellite_results/.DS_Store`,
`question_5_semantic_segmentation/outputs/satellite_results/P0362`,
`question_5_semantic_segmentation/outputs/satellite_results/P0362/class_ids.png`,
`question_5_semantic_segmentation/outputs/satellite_results/P0362/mask.png`

Sample tracked runs: `question_5_semantic_segmentation/runs/segment`, `question_5_semantic_segmentation/runs/segment/train`,
`question_5_semantic_segmentation/runs/segment/train/isaidd_yolo11s_seg`,
`question_5_semantic_segmentation/runs/segment/train/isaidd_yolo11s_seg/args.yaml`,
`question_5_semantic_segmentation/runs/segment/train/isaidd_yolo11s_seg/labels.jpg`,
`question_5_semantic_segmentation/runs/segment/train/isaidd_yolo11s_seg/train_batch0.jpg`

Sample tracked datasets: `question_5_semantic_segmentation/datasets/DOTAv1`,
`question_5_semantic_segmentation/datasets/DOTAv1/images`, `question_5_semantic_segmentation/datasets/DOTAv1/images/test`,
`question_5_semantic_segmentation/datasets/DOTAv1/images/test/P0006.jpg`,
`question_5_semantic_segmentation/datasets/DOTAv1/images/test/P0009.jpg`,
`question_5_semantic_segmentation/datasets/DOTAv1/images/test/P0014.jpg`

Original repo logic vs upstream backbone logic

- Repo-owned logic here is mostly the tracked `.py/.sh` wrapper layer plus any artifact shaping, environment enforcement, and summary-writing logic called out below.
- Calls `ultralytics.data.converter.convert_coco` inside the local Ultralytics environment to transform normalized COCO annotations into YOLO segmentation labels.
- Instantiates `ultralytics.YOLO` for inference; the repo-owned part starts after the raw result object comes back and turns it into semantic artifacts.
- Provides the Q5-owned glue code that sits in front of the upstream `external/ultralytics` package.
- Uses `network/download` helpers and then writes artifacts into the layout expected by Ultralytics segmentation training.
- Clones/reuses `external/ultralytics` and installs that local source tree; the optional `auto-bootstrap` then invokes the repo-owned dataset wrapper.
- Instantiates `ultralytics.YOLO` and subclasses `ultralytics.models.yolo.segment.train.SegmentationTrainer` so the actual optimization loop remains upstream.

What to explain deeply vs what to mention briefly

- Explain deeply: Explain `normalize_isaid_annotation_json(...)`, `prepare_isaid_raw_layout(...)`, and `write_dataset_yaml(...)` carefully; they are central repo-specific transformations, not generic upstream behavior.
- Explain deeply: Explain the nested `OSCSafeSegmentationTrainer` class because it shows the repo making a concrete engineering adaptation for OSC memory behavior.
- Explain deeply: Explain `extract_instances(...)`, `build_semantic_map(...)`, and `render_outputs(...)` because that is where the repo turns instance-style model outputs into the semantic artifacts expected by the assignment.
- Mention briefly: Mention raw Ultralytics optimizer internals briefly unless you reviewed the external code in detail.
- Mention briefly: Mention packaging-version checks in setup briefly; they are important for reliability but not the conceptual center of Q5.

Minimal run path

- Setup + optional auto-bootstrap: `bash question_5_semantic_segmentation/setup_yolo11_osc.sh`
- Manual bootstrap: `external/ultralytics/.venv/bin/python question_5_semantic_segmentation/bootstrap_isaid_seg.py --repo-dir external/ultralytics --download-dataset`
- Training: `bash osc_gpu_batch.sh --account <OSC_ACCOUNT> --time 04:00:00 -- external/ultralytics/.venv/bin/python question_5_semantic_segmentation/train_isaid_seg.py --repo-dir external/ultralytics --device 0 --epochs 20 --imgsz 768 --batch 1 --workers 0 --close-mosaic 20 --exist-ok`
- Inference: `external/ultralytics/.venv/bin/python question_5_semantic_segmentation/demo_yolo_segmentation.py --repo-dir external/ultralytics --device cpu`

Per-file walkthrough

File: `question_5_semantic_segmentation/README.md`

Classification: Repo-owned documentation/evidence file. Evidence anchor:
`question_5_semantic_segmentation/README.md:1-234`

Purpose: Acts as explicit repo evidence for workflow intent, minimal run steps, and ownership boundaries.

Key headings present in the tracked README: Semantic Segmentation with YOLO11 and iSAID, Overview, Setup, Dataset Bootstrap, Training, Inference, Inputs and Outputs, Environment Notes

File: `question_5_semantic_segmentation/bootstrap_isaid_seg.py`

Classification: Repo-owned dataset bootstrap and normalization wrapper. Evidence anchor:
`question_5_semantic_segmentation/bootstrap_isaid_seg.py:1-293`

Purpose: Turns raw iSAID and DOTA inputs into a normalized raw layout, YOLO segmentation labels, and a reusable dataset YAML and pretrained checkpoint.

- Upstream handoff: Calls `ultralalytics.data.converter.convert_coco` inside the local Ultralytics environment to transform normalized COCO annotations into YOLO segmentation labels.

Top import evidence: `__future__::annotations, argparse, sys, q5_seg_common::DEFAULT_ANNOTATIONS_DIR, DEFAULT_CONVERTED_DIR, DEFAULT_DATASET_YAML, DEFAULT_DOTA_EXTRACT_DIR, DEFAULT_DOTA_URL, DEFAULT_DOTA_ZIP, DEFAULT_ISAID_DATASET_PAGE_URL, DEFAULT_ISAID_GDRIVE_DIR, DEFAULT_PRETRAINED_MODEL, DEFAULT_RAW_ROOT, DEFAULT_REPO_DIR, DEFAULT_TRAIN_JSON, DEFAULT_VAL_JSON, DEFAULT_YOLO11_SEG_URL, DEFAULT_LABELS_DIR, download_file, ensure_ultralalytics_import, ensure_ultralalytics_repo, extract_zip, find_annotation_json, find_dota_root, gdown_download_folder, load_and_validate_category_maps, maybe_reexec_with_repo_python, normalize_isaid_annotation_json, parse_isaid_google_drive_links, prepare_isaid_raw_layout, replace_label_tree, resolve_question_path, resolve_repo_dir, validate_isaid_root, write_dataset_yaml`

Function-by-function walkthrough

build_parser (function; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:49-131`)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: `question_5_semantic_segmentation/bootstrap_isaid_seg.py:50-131`: Single contiguous control block.

main (function; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:134-280`)

Category: entrypoint orchestration. Summary: Owns the full Q5 data bootstrap narrative: path resolution, optional downloads, normalization, label conversion, YAML writing, and pretrained checkpoint fetch.

Line-block walk: `question_5_semantic_segmentation/bootstrap_isaid_seg.py:139-140`: Resolve the upstream repository path before touching the environment.; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:142-148`: Resolve all repository-local paths once so every later step uses the same values.; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:150-151`: Derive the normalized annotation and label directories from the raw root.; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:153-155`: Derive the normalized train and validation annotation file paths.; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:157-170`: Dry-run mode stops after printing the resolved paths.; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:172-173`: Re-exec inside the repository-local environment if the current Python differs.; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:175-177`: Import Ultralytics only after the correct environment is active.; `question_5_semantic_segmentation/bootstrap_isaid_seg.py:179-181`: Download the DOTA image archive first because the iSAID image splits reference it.

File: `question_5_semantic_segmentation/demo_yolo_segmentation.py`

Classification: Repo-owned inference/post-processing/reporting wrapper. Evidence anchor: `question_5_semantic_segmentation/demo_yolo_segmentation.py:1-777`

Purpose: Runs segmentation inference and converts instance predictions into fused semantic outputs, per-image summaries, and a batch index CSV.

- Upstream handoff: Instantiates `ultralalytics.YOLO` for inference; the repo-owned part starts after the raw result object comes back and turns it into semantic artifacts.

Top import evidence: `__future__::annotations, argparse, csv, json, sys, collections::Counter, pathlib::Path, q5_seg_common::DEFAULT_FINETUNED_MODEL, DEFAULT_INDEX_CSV, DEFAULT_INPUT_DIR, DEFAULT_OUTPUT_ROOT, DEFAULT_PRETRAINED_MODEL, DEFAULT_REPO_DIR, class_color, ensure_gpu_device_ready, ensure_parent, ensure_ultralalytics_import, ensure_ultralalytics_repo, is_url, list_supported_images, maybe_reexec_with_repo_python, normalize_name_map, parse_keep_classes, per_image_output_dir, resolve_model_argument, resolve_question_path, resolve_repo_dir, resolve_source, url_output_suffix`

Function-by-function walkthrough

build_parser (function; `question_5_semantic_segmentation/demo_yolo_segmentation.py:43-127`)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: `question_5_semantic_segmentation/demo_yolo_segmentation.py:44-127`: Single contiguous control block.

resolve_model_for_demo (function; `question_5_semantic_segmentation/demo_yolo_segmentation.py:130-149`)

Category: path/config resolution. Summary: Resolve the model input used for inference.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: `question_5_semantic_segmentation/demo_yolo_segmentation.py:134-137`: Resolve the requested model before importing YOLO. Resolve the requested model into the exact value YOLO should load.; `question_5_semantic_segmentation/demo_yolo_segmentation.py:139-149`: For local paths, fail early if the checkpoint is missing.

default_single_outputs (function; `question_5_semantic_segmentation/demo_yolo_segmentation.py:152-169`)

Category: local helper logic. Summary: Build the default output paths for one source.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:155: Derive one stable output stem from the local filename or URL.; question_5_semantic_segmentation/demo_yolo_segmentation.py:157-159: Reuse the local file stem and suffix for overlay naming.; question_5_semantic_segmentation/demo_yolo_segmentation.py:161-163: Use a generic stem for URLs and preserve any usable suffix.; question_5_semantic_segmentation/demo_yolo_segmentation.py:165-169: Single-image mode writes four sibling artifacts.

write_json (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:172-180)

Category: serialization/output writing. Summary: Write one JSON file.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:175: Create the parent directory and write one formatted JSON file.; question_5_semantic_segmentation/demo_yolo_segmentation.py:177: Make sure the output folder exists before writing.; question_5_semantic_segmentation/demo_yolo_segmentation.py:179-180: Write indented JSON so the summary stays easy to read.

write_csv (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:183-212)

Category: serialization/output writing. Summary: Write one CSV file.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:186: Flatten the per-image summaries into one batch index file.; question_5_semantic_segmentation/demo_yolo_segmentation.py:188-206: Create the destination folder before opening the CSV file.; question_5_semantic_segmentation/demo_yolo_segmentation.py:208-209: Write the header row first.; question_5_semantic_segmentation/demo_yolo_segmentation.py:211-212: Then write one row per image summary.

row_from_summary (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:215-244)

Category: local helper logic. Summary: Convert one summary into a CSV row.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:219-222: Pull the summary fields into the fixed CSV schema. Read the optional image shape from the summary payload.; question_5_semantic_segmentation/demo_yolo_segmentation.py:224-226: Clear the detection count on error rows.; question_5_semantic_segmentation/demo_yolo_segmentation.py:228-244: Return one flat row that matches the CSV schema exactly.

build_predict_kwargs (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:247-262)

Category: configuration or artifact construction. Summary: Build the YOLO predict options.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:249-258: Collect the predictor options once and pass them through to every image.; question_5_semantic_segmentation/demo_yolo_segmentation.py:260-262: Pass the device only when it was set explicitly.

extract_instances (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:265-306)

Category: structured extraction from upstream outputs. Summary: Turns the raw Ultralytics result object into a repo-owned detection list with class ids, confidences, boxes, pixel counts, and boolean masks.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:267: A segmentation result needs both boxes and masks to produce semantic outputs.; question_5_semantic_segmentation/demo_yolo_segmentation.py:269-271: Return no detections when either boxes or masks are missing.; question_5_semantic_segmentation/demo_yolo_segmentation.py:273-279: Move the model tensors to CPU numpy arrays for filtering and serialization.; question_5_semantic_segmentation/demo_yolo_segmentation.py:281-283: Skip detections outside the requested class filter.; question_5_semantic_segmentation/demo_yolo_segmentation.py:285-286: Convert the predicted mask into a boolean map.; question_5_semantic_segmentation/demo_yolo_segmentation.py:288-290: Skip empty masks after thresholding.; question_5_semantic_segmentation/demo_yolo_segmentation.py:292-303: Keep the fields needed for rendering, summaries, and per-pixel fusion.; question_5_semantic_segmentation/demo_yolo_segmentation.py:305-306: Sort by confidence so later stages can process the strongest detections first.

build_semantic_map (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:309-326)

Category: configuration or artifact construction. Summary: Resolves overlapping instances by per-pixel confidence competition, which is the key step that makes the final output semantic rather than merely instance-level.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:311: Start with background everywhere.; question_5_semantic_segmentation/demo_yolo_segmentation.py:313: Allocate the output class map with background id 0.; question_5_semantic_segmentation/demo_yolo_segmentation.py:315-317: Track the strongest confidence assigned to each pixel so overlaps can be resolved.; question_5_semantic_segmentation/demo_yolo_segmentation.py:319-320: Let the highest-confidence detection own each overlapping pixel.; question_5_semantic_segmentation/demo_yolo_segmentation.py:322: Write the class id anywhere this detection wins the overlap test.; question_5_semantic_segmentation/demo_yolo_segmentation.py:324-326: Update the winning confidence map at the same pixels.

colorize_class_map (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:329-336)

Category: local helper logic. Summary: Colorize the semantic class map.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:331-333: Expand the one-channel class map into a color image for visualization.; question_5_semantic_segmentation/demo_yolo_segmentation.py:335-336: Paint every pixel that belongs to the current class with its display color.

render_outputs (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:339-389)

Category: visualization/output rendering. Summary: Converts the fused class map into the three visible artifacts: overlay, raw class-id PNG, and colorized semantic mask.

Failure signals raised directly here: RuntimeError

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:341-342: Build the color mask first, then blend it with the original image.; question_5_semantic_segmentation/demo_yolo_segmentation.py:344-346: Blend the color mask over the source image to create the overlay.; question_5_semantic_segmentation/demo_yolo_segmentation.py:348-349: Draw one box and text label for each retained detection.; question_5_semantic_segmentation/demo_yolo_segmentation.py:351-352: Draw the bounding box first.; question_5_semantic_segmentation/demo_yolo_segmentation.py:354-358: Measure the label text so the background box fits it.; question_5_semantic_segmentation/demo_yolo_segmentation.py:360-366: Draw the filled text background box.; question_5_semantic_segmentation/demo_yolo_segmentation.py:368-378: Draw the label text on top of the filled background.; question_5_semantic_segmentation/demo_yolo_segmentation.py:380-382: Make sure every destination directory exists before writing the images.

classes_present (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:392-406)

Category: local helper logic. Summary: List the classes that appear in the class map.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:394-396: Report each non-background class id that appears in the semantic map.; question_5_semantic_segmentation/demo_yolo_segmentation.py:398-406: Convert the stored output-class id back to the model's zero-based class id.

pixel_counts_by_class (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:409-418)

Category: local helper logic. Summary: Count the pixels for each class.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:411-413: Count how many pixels belong to each class in the final semantic map.; question_5_semantic_segmentation/demo_yolo_segmentation.py:415: Convert the stored output-class id back to the model class id.; question_5_semantic_segmentation/demo_yolo_segmentation.py:417-418: Count pixels for this class in the final fused class map.

serializable_detections (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:421-435)

Category: local helper logic. Summary: Drop raw mask arrays from the detections.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:423: Drop the raw boolean mask arrays before writing detections into JSON.; question_5_semantic_segmentation/demo_yolo_segmentation.py:425-435: Keep only JSON-friendly scalar fields for each detection.

success_summary (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:438-463)

Category: local helper logic. Summary: Serializes the semantic result into a defensible JSON record with classes, pixel counts, instance counts, paths, and flattened detection metadata.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:440: Build one success payload with counts, artifact paths, and per-instance details.; question_5_semantic_segmentation/demo_yolo_segmentation.py:442: Count how many detections survived for each class name.; question_5_semantic_segmentation/demo_yolo_segmentation.py:444-463: Build the output-class-id to class-name map used by the saved class map.

error_summary (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:466-488)

Category: local helper logic. Summary: Build the error summary payload.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:468: Keep the error payload in the same shape as the success payload.; question_5_semantic_segmentation/demo_yolo_segmentation.py:470-488: Return the same keys as the success payload with empty artifact fields.

run_predict (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:491-499)

Category: stage execution. Summary: Run one predict call and return the first result.

Failure signals raised directly here: RuntimeError

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:495-497: Ultralytics returns a list of results even for one source. Run predict() once with the prepared keyword arguments.; question_5_semantic_segmentation/demo_yolo_segmentation.py:499: Use the first result because this wrapper passes one source at a time.

process_one_source (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:502-539)

Category: local helper logic. Summary: Process one source image from prediction to outputs.

Failure signals raised directly here: RuntimeError

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:504-506: Run one prediction and pull the original image back out of the result object.; question_5_semantic_segmentation/demo_yolo_segmentation.py:508-510: Fail if Ultralytics did not preserve the original image array.; question_5_semantic_segmentation/demo_yolo_segmentation.py:512-524: Convert detections into a per-pixel semantic map and write the image artifacts.; question_5_semantic_segmentation/demo_yolo_segmentation.py:526-539: Return the success payload after all image artifacts have been written.

run_single_mode (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:542-578)

Category: stage execution. Summary: Run the single-image flow.

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:544-546: Start with the default output paths, then apply any explicit overrides.; question_5_semantic_segmentation/demo_yolo_segmentation.py:548-556: Apply explicit output overrides one by one.; question_5_semantic_segmentation/demo_yolo_segmentation.py:558-572: Process the image, then write the companion JSON summary.; question_5_semantic_segmentation/demo_yolo_segmentation.py:574-578: Print the four artifact locations.

run_directory_mode (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:581-647)

Category: stage execution. Summary: Shows how batch execution is made reviewable: each image gets its own output folder and summary, and the whole run is flattened into one index CSV.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:583-590: Collect every supported image under the source directory.; question_5_semantic_segmentation/demo_yolo_segmentation.py:592-602: Resolve the batch output locations once before the loop starts.; question_5_semantic_segmentation/demo_yolo_segmentation.py:604-610: Each image gets its own output subdirectory and summary file.; question_5_semantic_segmentation/demo_yolo_segmentation.py:612-626: Successful images produce all artifacts plus a success summary.; question_5_semantic_segmentation/demo_yolo_segmentation.py:628-636: Failed images still produce a summary row so the batch index stays complete.; question_5_semantic_segmentation/demo_yolo_segmentation.py:638: Write the per-image summary file regardless of success or failure.; question_5_semantic_segmentation/demo_yolo_segmentation.py:640-641: Convert that summary into one flat CSV row.; question_5_semantic_segmentation/demo_yolo_segmentation.py:643-647: Write the aggregate CSV after every per-image summary is finished.

main (function; question_5_semantic_segmentation/demo_yolo_segmentation.py:650-764)

Category: entrypoint orchestration. Summary: Run the segmentation inference flow.

Failure signals raised directly here: ValueError

Line-block walk: question_5_semantic_segmentation/demo_yolo_segmentation.py:656-663: Resolve the repo, model, and source paths before importing heavy dependencies.; question_5_semantic_segmentation/demo_yolo_segmentation.py:665: Directory mode uses one output root, while single-image mode uses explicit files.; question_5_semantic_segmentation/demo_yolo_segmentation.py:667-679: Reject single-image output flags when the source is a directory.; question_5_semantic_segmentation/demo_yolo_segmentation.py:681-690: Print the resolved execution plan without importing YOLO or OpenCV.; question_5_semantic_segmentation/demo_yolo_segmentation.py:692-709: Recompute the single-image defaults so dry-run can show the final file paths.; question_5_semantic_segmentation/demo_yolo_segmentation.py:711-722: Re-enter under the repository interpreter before importing runtime dependencies.; question_5_semantic_segmentation/demo_yolo_segmentation.py:724-728: Import the runtime libraries only after the interpreter is finalized.; question_5_semantic_segmentation/demo_yolo_segmentation.py:730-738: Load the model and resolve any requested class filter.

File: question_5_semantic_segmentation/q5_seg_common.py

Classification: Repo-owned shared implementation backbone for Q5 bootstrap, train, and demo flows. Evidence anchor: question_5_semantic_segmentation/q5_seg_common.py:1-927

Purpose: Centralizes path resolution, environment enforcement, data download utilities, iSAID normalization helpers, label-tree replacement, and YAML generation for Q5.

- Upstream handoff: Provides the Q5-owned glue code that sits in front of the upstream external/ultralytics package.
- Upstream handoff: Uses network/download helpers and then writes artifacts into the layout expected by Ultralytics segmentation training.

Top import evidence: __future__: annotations, importlib, json, os, re, shutil, subprocess, sys

Function-by-function walkthrough

class_color (function; question_5_semantic_segmentation/q5_seg_common.py:105-108)

Category: local helper logic. Summary: Return the display color for one class id.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:107-108: Cycle through one fixed palette so class colors stay stable across runs.

normalize_name_map (function; question_5_semantic_segmentation/q5_seg_common.py:111-116)

Category: data normalization. Summary: Normalize the model name map into one dict.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:113-116: Ultralytics may expose class names as either a dict or a list-like object.

parse_keep_classes (function; question_5_semantic_segmentation/q5_seg_common.py:119-141)

Category: argument or text parsing. Summary: Parse the requested class filter.

Failure signals raised directly here: ValueError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:121-124: An empty filter keeps every class.; question_5_semantic_segmentation/q5_seg_common.py:126-141: Accept either numeric ids or case-insensitive class names.

is_url (function; question_5_semantic_segmentation/q5_seg_common.py:144-147)

Category: local helper logic. Summary: Return True when the value looks like a URL.

resolve_question_path (function; question_5_semantic_segmentation/q5_seg_common.py:150-156)

Category: path/config resolution. Summary: Resolve a path under this module folder.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:152-156: Anchor relative paths to this module folder.

resolve_repo_dir (function; question_5_semantic_segmentation/q5_seg_common.py:159-172)

Category: path/config resolution. Summary: Resolve the upstream repository directory.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:162-165: Default to the checked-out Ultralytics clone when no override is given.; question_5_semantic_segmentation/q5_seg_common.py:167-172: Prefer an existing cwd-relative path before falling back to repo-relative.

resolve_python (function; question_5_semantic_segmentation/q5_seg_common.py:175-183)

Category: path/config resolution. Summary: Resolve the Python executable to use.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:178-183: Prefer the repository-local virtual environment when no override is provided.

device_requests_gpu (function; question_5_semantic_segmentation/q5_seg_common.py:186-191)

Category: local helper logic. Summary: Return True when the device selection asks for CUDA.

resolve_python_minor_version (function; question_5_semantic_segmentation/q5_seg_common.py:194-212)

Category: path/config resolution. Summary: Ask one interpreter for its major.minor version.

Failure signals raised directly here: RuntimeError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:195-212: Single contiguous control block.

runtime_cuda_available (function; question_5_semantic_segmentation/q5_seg_common.py:215-227)

Category: subprocess orchestration. Summary: Return True when the target runtime reports CUDA availability.

osc_gpu_batch_example (function; question_5_semantic_segmentation/q5_seg_common.py:230-244)

Category: local helper logic. Summary: Build one OSC GPU launcher example command.

ensure_gpu_device_ready (function; question_5_semantic_segmentation/q5_seg_common.py:247-259)

Category: validation or readiness gate. Summary: Stop early when CUDA was requested without a GPU-ready runtime.

Failure signals raised directly here: RuntimeError

ensure_supported_repo_python_version (function; question_5_semantic_segmentation/q5_seg_common.py:262-271)

Category: validation or readiness gate. Summary: Reject unsupported interpreters for the OSC segmentation workflow.

Failure signals raised directly here: RuntimeError

maybe_reexec_with_repo_python (function; question_5_semantic_segmentation/q5_seg_common.py:274-289)

Category: local helper logic. Summary: Re-run under the repository interpreter when needed.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:280-289: Re-exec once so later imports run under the repository interpreter.

ensure_ultralytics_import (function; question_5_semantic_segmentation/q5_seg_common.py:292-317)

Category: validation or readiness gate. Summary: Import Ultralytics from the active environment or source checkout.

Failure signals raised directly here: ModuleNotFoundError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:295-300: First try the current interpreter environment as-is.; question_5_semantic_segmentation/q5_seg_common.py:302-317: If the package is missing, add the source checkout to sys.path and try again.

ensure_ultralytics_repo (function; question_5_semantic_segmentation/q5_seg_common.py:320-330)

Category: validation or readiness gate. Summary: Check that the Ultralytics clone looks valid.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:322-330: Validate the checkout before any scripts try to import from it.

resolve_model_argument (function; question_5_semantic_segmentation/q5_seg_common.py:333-358)

Category: path/config resolution. Summary: Resolve the model argument into the value YOLO should load.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:336-340: With no override, prefer the fine-tuned checkpoint and then the pretrained fallback.; question_5_semantic_segmentation/q5_seg_common.py:342-348: URLs pass straight through to the downstream loader.; question_5_semantic_segmentation/q5_seg_common.py:350-352: Path-like strings are anchored to this module folder.; question_5_semantic_segmentation/q5_seg_common.py:354-358: Plain filenames first check this module folder, then fall through unchanged.

resolve_source (function; question_5_semantic_segmentation/q5_seg_common.py:361-390)

Category: path/config resolution. Summary: Resolve the input source into a path or URL.

Failure signals raised directly here: FileNotFoundError, ValueError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:363: URLs are left untouched and handled in single-image mode.; question_5_semantic_segmentation/q5_seg_common.py:365-367: Return URL sources unchanged and mark them as non-path inputs.; question_5_semantic_segmentation/q5_seg_common.py:369-372: Expand "~" before deciding how to anchor the path.; question_5_semantic_segmentation/q5_seg_common.py:374-378: Prefer an existing cwd-relative path before falling back to this module folder.; question_5_semantic_segmentation/q5_seg_common.py:380-382: Stop early if the resolved path does not exist.; question_5_semantic_segmentation/q5_seg_common.py:384-390: Reject non-image files early so later inference code sees only supported inputs.

list_supported_images (function; question_5_semantic_segmentation/q5_seg_common.py:393-404)

Category: local helper logic. Summary: List the supported images under the directory.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:395: Walk recursively so nested image folders are processed in one batch run.; question_5_semantic_segmentation/q5_seg_common.py:397-404: Return one sorted list of supported files under the directory tree.

per_image_output_dir (function; question_5_semantic_segmentation/q5_seg_common.py:407-413)

Category: local helper logic. Summary: Build the output folder for one image.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:409: Mirror the source tree under the output root without the file suffix.; question_5_semantic_segmentation/q5_seg_common.py:411: Compute the image path relative to the batch source root.; question_5_semantic_segmentation/q5_seg_common.py:413: Reuse that relative path as the output folder name without the extension.

url_output_suffix (function; question_5_semantic_segmentation/q5_seg_common.py:416-422)

Category: local helper logic. Summary: Pick a usable image suffix for a URL source.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:418: Preserve a supported URL suffix when one is present.; question_5_semantic_segmentation/q5_seg_common.py:420: Pull the suffix from the URL path only.; question_5_semantic_segmentation/q5_seg_common.py:422: Fall back to .jpg when the URL does not end in a supported image suffix.

ensure_parent (function; question_5_semantic_segmentation/q5_seg_common.py:425-430)

Category: validation or readiness gate. Summary: Create the parent directory for a file path.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:427: Create the parent directory before writing the file.; question_5_semantic_segmentation/q5_seg_common.py:429-430: Create every missing parent directory in one call.

human_bytes (function; question_5_semantic_segmentation/q5_seg_common.py:433-444)

Category: local helper logic. Summary: Format a byte count in a readable unit.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:435-438: Convert the raw byte count into the first readable unit.; question_5_semantic_segmentation/q5_seg_common.py:440-441: Stop once the value is small enough for the current unit.; question_5_semantic_segmentation/q5_seg_common.py:443-444: Otherwise move to the next larger unit.

download_file (function; question_5_semantic_segmentation/q5_seg_common.py:447-493)

Category: download or remote fetch. Summary: Download one file when it is missing.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:449: Reuse an existing download unless force requests a refresh.; question_5_semantic_segmentation/q5_seg_common.py:451-452: Resolve the destination and create its parent directory first.; question_5_semantic_segmentation/q5_seg_common.py:454-461: Reuse the existing file when force is not set.; question_5_semantic_segmentation/q5_seg_common.py:463-467: Stream into a temporary file so incomplete downloads never look valid.; question_5_semantic_segmentation/q5_seg_common.py:469-471: Read the next 1 MB chunk from the response.; question_5_semantic_segmentation/q5_seg_common.py:473-475: Append the chunk to the temporary file.; question_5_semantic_segmentation/q5_seg_common.py:477-482: Overwrite one progress line while the transfer is active.; question_5_semantic_segmentation/q5_seg_common.py:484: Clear the progress line after the transfer finishes.

ensure_clean_dir (function; question_5_semantic_segmentation/q5_seg_common.py:496-505)

Category: validation or readiness gate. Summary: Replace the directory with a clean empty one.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:498-499: Replace any previous directory contents with a new empty directory.; question_5_semantic_segmentation/q5_seg_common.py:501-502: Remove the old tree when it already exists.; question_5_semantic_segmentation/q5_seg_common.py:504-505: Recreate the directory in a clean state.

extract_zip (function; question_5_semantic_segmentation/q5_seg_common.py:508-520)

Category: structured extraction from upstream outputs. Summary: Extract one zip archive.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:510: Expand the archive into the requested destination directory.;
question_5_semantic_segmentation/q5_seg_common.py:512-513: Resolve both paths before extraction.;
question_5_semantic_segmentation/q5_seg_common.py:515-517: Make sure the destination folder exists first.;
question_5_semantic_segmentation/q5_seg_common.py:519-520: Unpack every file in the archive into the destination tree.

validate_dota_images_root (function; question_5_semantic_segmentation/q5_seg_common.py:523-533)

Category: validation or contract enforcement. Summary: Check that the DOTA image layout is complete.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:525-527: DOTA image bundles must provide both train and val image folders.; question_5_semantic_segmentation/q5_seg_common.py:529-533: Collect every missing required directory before raising.

find_dota_root (function; question_5_semantic_segmentation/q5_seg_common.py:536-559)

Category: local helper logic. Summary: Find the extracted DOTA root directory.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:538-542: Check the common extraction path first.;
question_5_semantic_segmentation/q5_seg_common.py:544-547: Return the common extraction root when its layout is complete.;
question_5_semantic_segmentation/q5_seg_common.py:549-551: Fall back to a recursive search if the archive unpacked into a different layout.;
question_5_semantic_segmentation/q5_seg_common.py:553-559: Stop at the first directory that looks like a valid DOTA root.

fetch_text (function; question_5_semantic_segmentation/q5_seg_common.py:562-568)

Category: download or remote fetch. Summary: Fetch a text response from a URL.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:564-566: Download one text payload with a browser-style UA header.;
question_5_semantic_segmentation/q5_seg_common.py:568: Decode the full response body as UTF-8 text.

parse_isaid_google_drive_links (function; question_5_semantic_segmentation/q5_seg_common.py:571-584)

Category: argument or text parsing. Summary: Parse the train and val Google Drive links.

Failure signals raised directly here: RuntimeError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:573-584: Pull the train and validation folder links from the public dataset page.

gdown_download_folder (function; question_5_semantic_segmentation/q5_seg_common.py:587-617)

Category: subprocess orchestration. Summary: Download one Google Drive folder when needed.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:589-590: Reuse an existing folder download unless force requests a refresh.; question_5_semantic_segmentation/q5_seg_common.py:592-595: Reuse a non-empty download folder unless force is set.;
question_5_semantic_segmentation/q5_seg_common.py:597-598: Remove the old folder when force requested a clean download.;
question_5_semantic_segmentation/q5_seg_common.py:600-601: Recreate the destination folder before running gdown.;
question_5_semantic_segmentation/q5_seg_common.py:603-612: Build the gdown command that downloads the full folder tree.;
question_5_semantic_segmentation/q5_seg_common.py:614: Delegate Google Drive folder handling to gdown instead of reimplementing it here.;
question_5_semantic_segmentation/q5_seg_common.py:616-617: Let gdown handle the transfer and fail loudly if it returns an error.

find_annotation_json (function; question_5_semantic_segmentation/q5_seg_common.py:620-655)

Category: local helper logic. Summary: Find the annotation JSON for the requested split.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:622-624: Gather every JSON file first so the later selection logic can be explicit.; question_5_semantic_segmentation/q5_seg_common.py:626-628: Stop immediately when the folder contains no JSON files at all.;
question_5_semantic_segmentation/q5_seg_common.py:630-637: Prefer filenames that mention the requested split and are not test annotations.;
question_5_semantic_segmentation/q5_seg_common.py:639-655: Prefer the most instance-like filename when several matches exist.

ensure_symlink_or_copy (function; question_5_semantic_segmentation/q5_seg_common.py:658-699)

Category: validation or readiness gate. Summary: Create a symlink or copy to the destination.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:660-662: Resolve the source once so later symlink checks compare canonical paths.; question_5_semantic_segmentation/q5_seg_common.py:664-667: Resolve relative destinations against the current working directory.; question_5_semantic_segmentation/q5_seg_common.py:669-679: Leave an existing correct symlink in place.;
question_5_semantic_segmentation/q5_seg_common.py:681-682: Remove an incorrect symlink before replacing it.;
question_5_semantic_segmentation/q5_seg_common.py:684-690: Remove an existing real file or directory before replacing it.;
question_5_semantic_segmentation/q5_seg_common.py:692-693: Prefer a symlink so the raw layout does not duplicate large image trees.;
question_5_semantic_segmentation/q5_seg_common.py:695-699: Fall back to a real copy on filesystems that do not allow symlinks.

copy_file (function; question_5_semantic_segmentation/q5_seg_common.py:702-711)

Category: artifact copying or dataset materialization. Summary: Copy one file to the destination.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:704-706: Copy one file after creating its parent directory.;
question_5_semantic_segmentation/q5_seg_common.py:708: Create the destination folder first.;
question_5_semantic_segmentation/q5_seg_common.py:710-711: Copy the file metadata and contents together.

canonicalize_isaid_name (function; question_5_semantic_segmentation/q5_seg_common.py:714-718)

Category: local helper logic. Summary: Normalize an iSAID class name.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:716-718: Normalize category names so different punctuation still maps to one class id.

find_matching_image_file (function; question_5_semantic_segmentation/q5_seg_common.py:721-737)

Category: local helper logic. Summary: Find the image file that matches the annotation name.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:723-725: Try the JSON filename first.;
question_5_semantic_segmentation/q5_seg_common.py:727-729: Return the exact filename match when it exists.;
question_5_semantic_segmentation/q5_seg_common.py:731-737: Then try the same stem with the common image suffixes used by iSAID/DOTA.

normalize_isaid_annotation_json (function; question_5_semantic_segmentation/q5_seg_common.py:740-795)

Category: data normalization. Summary: This is one of the most important original Q5 transforms: it rewrites category ids/names into the repo's canonical class space and fixes image filenames/size metadata when needed.

Failure signals raised directly here: ValueError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:742-744: Load the COCO annotation file before remapping any category ids.;
question_5_semantic_segmentation/q5_seg_common.py:746-748: Read the JSON payload into memory so it can be rewritten in place.;
question_5_semantic_segmentation/q5_seg_common.py:750-756: Pull out the two sections that need to be normalized.;
question_5_semantic_segmentation/q5_seg_common.py:758-768: Map each original category id onto the fixed contiguous iSAID id space.;
question_5_semantic_segmentation/q5_seg_common.py:770-776: Rewrite every annotation to the normalized category id.;
question_5_semantic_segmentation/q5_seg_common.py:778-780: Replace the filename with the actual RGB image name used on disk.;
question_5_semantic_segmentation/q5_seg_common.py:782-784: Fill in missing image size metadata directly from the file.;
question_5_semantic_segmentation/q5_seg_common.py:786-790: Rewrite the category table to the normalized training ids and names.

prepare_isaid_raw_layout (function; question_5_semantic_segmentation/q5_seg_common.py:798-817)

Category: local helper logic. Summary: Builds the raw dataset contract used by the rest of Q5 by linking/copying DOTA image trees and writing normalized annotation files into one stable layout.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:800-801: Link or copy the RGB image folders into the normalized raw layout.;
question_5_semantic_segmentation/q5_seg_common.py:803: Put the DOTA train images under raw_root/images/train.;
question_5_semantic_segmentation/q5_seg_common.py:805: Put the DOTA val images under raw_root/images/val.;
question_5_semantic_segmentation/q5_seg_common.py:807-817: Normalize both annotation files into the same raw layout.

validate_isaid_root (function; question_5_semantic_segmentation/q5_seg_common.py:820-838)

Category: validation or contract enforcement. Summary: Check that the normalized iSAID layout is complete.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:822-829: The normalized layout must contain both image splits and both annotation files.;
question_5_semantic_segmentation/q5_seg_common.py:831-838: Collect any missing required paths before raising one combined error.

load_coco_categories (function; question_5_semantic_segmentation/q5_seg_common.py:841-864)

Category: loading/ingest. Summary: Load the COCO categories from one annotation file.

Failure signals raised directly here: ValueError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:843-844: Load the category table from one COCO annotation file.;
question_5_semantic_segmentation/q5_seg_common.py:846-849: Read the full annotation JSON once.;
question_5_semantic_segmentation/q5_seg_common.py:851-853: The category table must exist and contain at least one class.;
question_5_semantic_segmentation/q5_seg_common.py:855-864: Sort categories by id before validating the numeric sequence.

load_and_validate_category_maps (function; question_5_semantic_segmentation/q5_seg_common.py:867-879)

Category: loading/ingest. Summary: Prevents silent train/val drift by requiring both splits to expose the same contiguous class map before YAML generation or training.

Failure signals raised directly here: ValueError

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:869: Require train and validation splits to expose the same class map.;
question_5_semantic_segmentation/q5_seg_common.py:871: Load the train split category map.;

question_5_semantic_segmentation/q5_seg_common.py:873: Load the validation split category map.;
question_5_semantic_segmentation/q5_seg_common.py:875-879: Stop if the two splits disagree on the class id mapping.

write_dataset_yaml (function; question_5_semantic_segmentation/q5_seg_common.py:882-902)

Category: serialization/output writing. Summary: Writes the exact Ultralytics dataset descriptor consumed by training, so this is the final repo-owned handoff from data engineering into the model stack.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:884-886: Write the compact dataset description that Ultralytics expects.;
question_5_semantic_segmentation/q5_seg_common.py:888-898: Create the parent directory before writing the YAML file.;
question_5_semantic_segmentation/q5_seg_common.py:900-902: Write one numeric class id to class name mapping per line.

replace_label_tree (function; question_5_semantic_segmentation/q5_seg_common.py:905-920)

Category: local helper logic. Summary: Replace the label tree with the new files.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:907-909: Replace the destination tree with the newly generated label files.;
question_5_semantic_segmentation/q5_seg_common.py:911-912: Remove any older label tree first.;
question_5_semantic_segmentation/q5_seg_common.py:914-916: Recreate the destination root before moving files into it.;
question_5_semantic_segmentation/q5_seg_common.py:918-920: Preserve the relative split/subfolder layout while moving the labels.

summarize_candidates (function; question_5_semantic_segmentation/q5_seg_common.py:923-927)

Category: summary/report formatting. Summary: Join candidate paths into one string.

Line-block walk: question_5_semantic_segmentation/q5_seg_common.py:925: Join candidate paths into one readable error string.;
question_5_semantic_segmentation/q5_seg_common.py:927: Resolve each path so the final message is unambiguous.

File: question_5_semantic_segmentation/setup_yolo11_osc.sh

Classification: Repo-owned environment bootstrap and optional dataset auto-bootstrap shell script. Evidence anchor:
question_5_semantic_segmentation/setup_yolo11_osc.sh:1-216

Purpose: Ensures the Ultralytics environment is valid and optionally bootstraps the dataset and pretrained checkpoint immediately.

- Upstream handoff: Clones/reuses external/ultralytics and installs that local source tree; the optional auto-bootstrap then invokes the repo-owned dataset wrapper.

Top-level shell stages

question_5_semantic_segmentation/setup_yolo11_osc.sh:2-3: !usr/bin/env bash

question_5_semantic_segmentation/setup_yolo11_osc.sh:5-4: Prepare the repo-local Ultralytics environment used by the Q5 segmentation

question_5_semantic_segmentation/setup_yolo11_osc.sh:6-5: wrappers. Unlike Q4, this setup may also bootstrap the dataset and pretrained

question_5_semantic_segmentation/setup_yolo11_osc.sh:7-25: checkpoint because the tracked Q5 CLI expects those artifacts to exist.

question_5_semantic_segmentation/setup_yolo11_osc.sh:27-29: Resolve relative clone targets before the rest of the script uses the path.

question_5_semantic_segmentation/setup_yolo11_osc.sh:31-30: Clone or reuse the upstream repository before rebuilding the environment.

question_5_semantic_segmentation/setup_yolo11_osc.sh:32-33: Create the parent directory before cloning into it.

question_5_semantic_segmentation/setup_yolo11_osc.sh:35-39: Stop immediately if the requested interpreter is unavailable.

question_5_semantic_segmentation/setup_yolo11_osc.sh:41-58: Reuse the existing clone when it is already present.

question_5_semantic_segmentation/setup_yolo11_osc.sh:60-76: Reuse the virtual environment when it already exists.

question_5_semantic_segmentation/setup_yolo11_osc.sh:78-79: Activate the environment before installing or checking packages.

question_5_semantic_segmentation/setup_yolo11_osc.sh:81-80: Keep the packaging bootstrap tools below versions known to work with this

question_5_semantic_segmentation/setup_yolo11_osc.sh:82-81: OSC-ready Ultralytics segmentation path.

question_5_semantic_segmentation/setup_yolo11_osc.sh:83-111: Verify the packaging tool versions before reinstalling them.

question_5_semantic_segmentation/setup_yolo11_osc.sh:113-112: Remove stale site-packages copies so imports resolve to this checkout rather

question_5_semantic_segmentation/setup_yolo11_osc.sh:114-113: than to any previously installed Ultralytics wheel.

Representative commands and side effects

```
set -euo pipefail
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

```
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
```

```
DEFAULT_REPO_DIR="${REPO_ROOT}/external/ultralytics"
```

```
REPO_URL="${REPO_URL:-https://github.com/ultralytics/ultralytics.git}"
REPO_DIR="${1:-${DEFAULT_REPO_DIR}}"
PYTHON_BIN="${PYTHON_BIN:-}"
SUPPORTED_PYTHON_VERSION="${SUPPORTED_PYTHON_VERSION:-3.9.18}"
```

File: question_5_semantic_segmentation/train_isaid_seg.py

Classification: Repo-owned training wrapper around Ultralytics segmentation. Evidence anchor: question_5_semantic_segmentation/train_isaid_seg.py:1-336

Purpose: Launches training with OSC-specific safeguards, resume handling, and stable output aliasing for the fine-tuned checkpoint.

- Upstream handoff: Instantiates `ultralytics.YOLO` and subclasses `ultralytics.models.yolo.segment.train.SegmentationTrainer` so the actual optimization loop remains upstream.

Top import evidence: `__future__::annotations, argparse, shutil, sys, pathlib::Path, q5_seg_common::DEFAULT_DATASET_YAML, DEFAULT_FINETUNED_MODEL, DEFAULT_PRETRAINED_MODEL, DEFAULT_REPO_DIR, DEFAULT_TRAIN_PROJECT_DIR, ensure_gpu_device_ready, ensure_parent, ensure_ultralytics_import, ensure_ultralytics_repo, is_url, maybe_reexec_with_repo_python, resolve_model_argument, resolve_question_path, resolve_repo_dir`

Function-by-function walkthrough

build_parser (function; question_5_semantic_segmentation/train_isaid_seg.py:33-108)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: question_5_semantic_segmentation/train_isaid_seg.py:34-108: Single contiguous control block.

resolve_training_model (function; question_5_semantic_segmentation/train_isaid_seg.py:111-125)

Category: path/config resolution. Summary: Fails early when the requested checkpoint is missing, which keeps training errors attributable to setup rather than buried deep in Ultralytics.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: question_5_semantic_segmentation/train_isaid_seg.py:114-117: Resolve the requested model into the exact value `YOLO` should load; question_5_semantic_segmentation/train_isaid_seg.py:119-125: Fail early if the local checkpoint path does not exist.

main (function; question_5_semantic_segmentation/train_isaid_seg.py:128-323)

Category: entrypoint orchestration. Summary: Stages the training run, guards resume behavior, loads the model, injects the custom trainer, and copies the chosen checkpoint to a stable alias path.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: question_5_semantic_segmentation/train_isaid_seg.py:133-134: Resolve the upstream repo path before touching the environment; question_5_semantic_segmentation/train_isaid_seg.py:136-143: Resolve the shared paths once so later steps reuse the same values; question_5_semantic_segmentation/train_isaid_seg.py:145-163: Dry-run mode stops after printing the resolved paths.; question_5_semantic_segmentation/train_isaid_seg.py:165-176: Re-exec inside the repository-local environment if the current Python differs.; question_5_semantic_segmentation/train_isaid_seg.py:178-179: Import Ultralytics only after the correct environment is active.; question_5_semantic_segmentation/train_isaid_seg.py:181-277: Stop before training if the prepared dataset YAML is missing.; question_5_semantic_segmentation/train_isaid_seg.py:279-280: Load the starting checkpoint into a YOLO model object.; question_5_semantic_segmentation/train_isaid_seg.py:282-295: Build the keyword arguments passed into the Ultralytics trainer.

main.load_trusted_checkpoint (function; question_5_semantic_segmentation/train_isaid_seg.py:195-200)

Category: loading/ingest. Summary: Load one locally generated training checkpoint across Torch defaults.

main.completed_epochs (function; question_5_semantic_segmentation/train_isaid_seg.py:202-210)

Category: local helper logic. Summary: Return the 1-based completed epoch count stored in one checkpoint.

main.OSCSafeSegmentationTrainer (class; question_5_semantic_segmentation/train_isaid_seg.py:226-260)

Category: class/type definition. Summary: This nested subclass is the core OSC-specific training adaptation: it reduces validation loader pressure and suppresses redundant final validation behavior.

Line-block walk: question_5_semantic_segmentation/train_isaid_seg.py:227-260: Single contiguous control block.

main.OSCSafeSegmentationTrainer.check_resume (function; question_5_semantic_segmentation/train_isaid_seg.py:229-233)

Category: local helper logic. Summary: Keep the requested epoch count when resuming training.

main.OSCSafeSegmentationTrainer._build_train_pipeline (function; question_5_semantic_segmentation/train_isaid_seg.py:235-249)

Category: local helper logic. Summary: Rebuild the validation loader with a smaller OSC-safe batch size.

main.OSCSafeSegmentationTrainer.final_eval (function;
question_5_semantic_segmentation/train_isaid_seg.py:251-260)

Category: local helper logic. Summary: Skip Ultralytics' redundant post-training best.pt validation on OSC.

Not found in repo

- A tracked benchmark of segmentation quality on iSAID is not found in repo.
- A formal explanation for why this exact 15-class palette was chosen is not found in repo beyond the fixed mapping encoded in `q5_seg_common.py`.

Q6 Directional Convolution Filters

Tracked scope: 4 top-level files, 766 lines, 26 Python definitions, 0 shell functions.

What it is

- The most purely original algorithmic folder in the repository: six directional kernels are derived in local code, then applied to both synthetic and real-image inputs.
- There is no external deep-learning backbone here; the folder's value is that every step is inspectable, mathematically small, and easy to defend line by line.

Why this folder exists in the repo

- To demonstrate understanding of orientation-sensitive convolution without hiding behind a pretrained model.
- To produce both structured numeric output and visual artifacts that make the filter responses defensible.

Architecture and data flow

- The library file defines the direction vectors, kernel derivation, same-size zero-padded convolution, absolute-response normalization, and summary statistics.
- The synthetic demo builds a test image with horizontal, vertical, and diagonal strokes, runs all kernels, prints the results, and writes a JSON summary.
- The real-image demo converts RGB to grayscale, stretches contrast, thresholds to black/white, applies the same filters, normalizes each response to uint8, writes individual maps, and assembles a 2x4 contact sheet.

Tracked files and responsibilities

question_6_convolution_filters/README.md -> Repo-owned documentation/evidence file. (73 lines).

question_6_convolution_filters/convolution_filter_library.py -> Repo-owned implementation of the directional kernels and convolution math. (188 lines).

question_6_convolution_filters/demo_convolution_filters.py -> Repo-owned synthetic demonstration and JSON reporting script. (142 lines).

question_6_convolution_filters/demo_convolution_filters_image.py -> Repo-owned real-image preprocessing, filtering, visualization, and reporting script. (363 lines).

Data transformations and artifact lifecycle

- Artifact counts now present on disk: outputs 61, runs 0, datasets 0, logs 0.
- Synthetic demo JSON exists and records kernels: horizontal, vertical, upper_left, lower_right, upper_right, lower_left.
- Tracked response stats include fields like min, max, mean_abs, and sum_abs for horizontal.
- Per-image output directories present: 6.

Sample tracked outputs: question_6_convolution_filters/outputs/000000000139,
question_6_convolution_filters/outputs/000000000139/000000000139_feature_maps_grid.png,
question_6_convolution_filters/outputs/000000000139/000000000139_horizontal.png,
question_6_convolution_filters/outputs/000000000139/000000000139_lower_left.png,
question_6_convolution_filters/outputs/000000000139/000000000139_lower_right.png,
question_6_convolution_filters/outputs/000000000139/000000000139_preprocessed_bw.png

Original repo logic vs upstream backbone logic

- Repo-owned logic here is mostly the tracked .py/.sh wrapper layer plus any artifact shaping, environment enforcement, and summary-writing logic called out below.
- No upstream model backbone is used here; this folder is the clearest example of code that is original to this repository.
- No external backbone beyond the local convolution_filter_library.py module.
- Uses Pillow for image IO, but the convolution and reporting logic stays in repo-owned code.

What to explain deeply vs what to mention briefly

- Explain deeply: Explain derive_directional_kernel(...) and convolve2d(...) in detail; they are the core logic most worth defending line by line.

- Explain deeply: Explain why the preprocessing pipeline is RGB -> grayscale -> autocontrast -> threshold at 128 before convolution.
- Explain deeply: Explain how `normalize_matrix_abs_to_uint8(...)` and `matrix_stats(...)` make the feature maps both viewable and comparable.
- Mention briefly: Mention Pillow-only IO helpers briefly; they support the pipeline but are not the algorithmic heart.
- Mention briefly: Mention CLI argument parsing briefly unless asked about interface design.

Minimal run path

- Synthetic demo: `python3.9 question_6_convolution_filters/demo_convolution_filters.py --size 11`
- Image demo: `python3.9 question_6_convolution_filters/demo_convolution_filters_image.py --source question_6_convolution_filters/inputs --output-dir question_6_convolution_filters/outputs`

Per-file walkthrough

File: `question_6_convolution_filters/README.md`

Classification: Repo-owned documentation/evidence file. Evidence anchor: `question_6_convolution_filters/README.md:1-73`

Purpose: Acts as explicit repo evidence for workflow intent, minimal run steps, and ownership boundaries.

Key headings present in the tracked README: Directional Convolution Filters, Overview, Setup, Run, Inputs and Outputs

File: `question_6_convolution_filters/convolution_filter_library.py`

Classification: Repo-owned implementation of the directional kernels and convolution math. Evidence anchor: `question_6_convolution_filters/convolution_filter_library.py:1-188`

Purpose: Implements the actual kernel derivation, convolution, normalization, and response statistics without relying on an external deep-learning framework.

- Upstream handoff: No upstream model backbone is used here; this folder is the clearest example of code that is original to this repository.

Top import evidence: `__future__::annotations`

Function-by-function walkthrough

`_sign` (function; `question_6_convolution_filters/convolution_filter_library.py:23-33`)

Category: local helper logic. Summary: Return the sign of one numeric value.

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:27-28`: Reduce every input to the `{-1, 0, 1}` range used by the kernels. Positive values map to 1; `question_6_convolution_filters/convolution_filter_library.py:30-31`: Negative values map to -1; `question_6_convolution_filters/convolution_filter_library.py:33`: Zero stays zero.

`_normalize_direction` (function; `question_6_convolution_filters/convolution_filter_library.py:36-47`)

Category: local helper logic. Summary: Clamp a direction vector to single-step components.

Failure signals raised directly here: `ValueError`

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:40`: Normalize each component before the kernel builder uses it. Unpack the input direction tuple.; `question_6_convolution_filters/convolution_filter_library.py:42-43`: Reduce each component to -1, 0, or 1; `question_6_convolution_filters/convolution_filter_library.py:45-47`: Reject the zero vector because it has no direction.

`derive_directional_kernel` (function; `question_6_convolution_filters/convolution_filter_library.py:50-77`)

Category: local helper logic. Summary: The cleanest algorithmic core in the repo: project each kernel cell onto the target direction and reduce that projection to -1, 0, or 1.

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:53-55`: Normalize the requested direction before building the kernel.; `question_6_convolution_filters/convolution_filter_library.py:57-58`: Walk the 3x3 window row by row.; `question_6_convolution_filters/convolution_filter_library.py:60`: Project the current cell onto the requested direction.; `question_6_convolution_filters/convolution_filter_library.py:62`: Convert the projection back to -1, 0, or 1.; `question_6_convolution_filters/convolution_filter_library.py:64-65`: Finish the current kernel row.; `question_6_convolution_filters/convolution_filter_library.py:67-69`: Return the raw signed kernel unless normalized weights were requested.; `question_6_convolution_filters/convolution_filter_library.py:71`: Scale the positive side so different kernels share the same total weight.; `question_6_convolution_filters/convolution_filter_library.py:73-75`: Leave the kernel unchanged if it has no positive weights.

`get_directional_kernels` (function; `question_6_convolution_filters/convolution_filter_library.py:80-86`)

Category: local helper logic. Summary: Return horizontal, vertical, and 4 diagonal kernels.

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:83-86`: Build one named kernel for each predefined direction.

convolve2d (function; `question_6_convolution_filters/convolution_filter_library.py:90-141`)

Category: local helper logic. Summary: Implements same-size zero-padded convolution explicitly, making the spatial math defensible without relying on any framework primitive.

Failure signals raised directly here: `ValueError`

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:93-97`: Reject empty inputs before any shape math runs.; `question_6_convolution_filters/convolution_filter_library.py:99-101`: Capture the image shape once and verify the rows are consistent.; `question_6_convolution_filters/convolution_filter_library.py:103-105`: Every image row must match the first row width.; `question_6_convolution_filters/convolution_filter_library.py:107-109`: Capture the kernel shape once and verify the rows are consistent.; `question_6_convolution_filters/convolution_filter_library.py:111-113`: Every kernel row must match the first row width.; `question_6_convolution_filters/convolution_filter_library.py:115-117`: Same-size convolution needs odd kernel dimensions so there is a true center.; `question_6_convolution_filters/convolution_filter_library.py:119-121`: Use the half-width of the kernel to center it over each output pixel.; `question_6_convolution_filters/convolution_filter_library.py:123-126`: Allocate one output value per input pixel.

apply_all_directional_filters (function; `question_6_convolution_filters/convolution_filter_library.py:144-149`)

Category: local helper logic. Summary: Apply all 6 directional filters to an image.

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:147`: Build the kernel set once, then run one convolution per direction.; `question_6_convolution_filters/convolution_filter_library.py:149`: Convolve the image with every named kernel.

normalize_matrix_abs_to_uint8 (function; `question_6_convolution_filters/convolution_filter_library.py:152-174`)

Category: data normalization. Summary: Scale absolute matrix responses independently into the 0-255 range.

Failure signals raised directly here: `ValueError`

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:160-162`: Every row must match the first row width.; `question_6_convolution_filters/convolution_filter_library.py:164-167`: Keep an all-zero matrix all-zero instead of dividing by zero.; `question_6_convolution_filters/convolution_filter_library.py:169`: Scale by the strongest absolute response so the image uses the full range.; `question_6_convolution_filters/convolution_filter_library.py:171-174`: Convert every absolute response into an 8-bit intensity.

matrix_stats (function; `question_6_convolution_filters/convolution_filter_library.py:177-188`)

Category: local helper logic. Summary: Compute summary statistics for one response matrix.

Line-block walk: `question_6_convolution_filters/convolution_filter_library.py:180`: Flatten the matrix once so the summary metrics reuse the same values.; `question_6_convolution_filters/convolution_filter_library.py:182-188`: Build the absolute-value view used by the magnitude stats.

File: `question_6_convolution_filters/demo_convolution_filters.py`

Classification: Repo-owned synthetic demonstration and JSON reporting script. Evidence anchor: `question_6_convolution_filters/demo_convolution_filters.py:1-142`

Purpose: Builds a synthetic image whose orientations make the kernel behavior easy to justify and compare.

- Upstream handoff: No external backbone beyond the `local_convolution_filter_library.py` module.

Top import evidence: `__future__::annotations`, `argparse`, `json`, `pathlib::Path`, `convolution_filter_library::apply_all_directional_filters`, `get_directional_kernels`, `matrix_stats`

Function-by-function walkthrough

build_synthetic_image (function; `question_6_convolution_filters/demo_convolution_filters.py:23-47`)

Category: configuration or artifact construction. Summary: Constructs a test pattern that guarantees all six orientations are represented, which is why the response summary is easy to justify.

Failure signals raised directly here: `ValueError`

Line-block walk: `question_6_convolution_filters/demo_convolution_filters.py:26-28`: Reject sizes that cannot place the center-aligned patterns cleanly.; `question_6_convolution_filters/demo_convolution_filters.py:30-32`: Start from an all-zero image.; `question_6_convolution_filters/demo_convolution_filters.py:34-36`: Draw the horizontal stroke through the center row.; `question_6_convolution_filters/demo_convolution_filters.py:38-40`: Draw the vertical stroke through the center column.; `question_6_convolution_filters/demo_convolution_filters.py:42-45`: Draw both diagonals so every kernel sees one matching structure.; `question_6_convolution_filters/demo_convolution_filters.py:47`: Return the completed synthetic image matrix.

format_matrix (function; `question_6_convolution_filters/demo_convolution_filters.py:50-57`)

Category: summary/report formatting. Summary: Format one matrix as aligned text.

Line-block walk: `question_6_convolution_filters/demo_convolution_filters.py:53-54`: Format each row to a fixed width so the grid stays readable.; `question_6_convolution_filters/demo_convolution_filters.py:56-57`: Format one row of numeric values into aligned columns.

summarize_responses (function; `question_6_convolution_filters/demo_convolution_filters.py:60-63`)

Category: summary/report formatting. Summary: Reduce each full response map to summary metrics.

Line-block walk: `question_6_convolution_filters/demo_convolution_filters.py:63`: Run the shared stats helper on each named response matrix.

build_parser (function; `question_6_convolution_filters/demo_convolution_filters.py:66-86`)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: `question_6_convolution_filters/demo_convolution_filters.py:67-86`: Single contiguous control block.

main (function; `question_6_convolution_filters/demo_convolution_filters.py:89-138`)

Category: entrypoint orchestration. Summary: Run the synthetic-image filter comparison.

Line-block walk: `question_6_convolution_filters/demo_convolution_filters.py:92-93`: Parse the CLI configuration once at startup.; `question_6_convolution_filters/demo_convolution_filters.py:95-96`: Build the input image before deriving kernels or responses.; `question_6_convolution_filters/demo_convolution_filters.py:98-107`: Derive the kernels and evaluate them against the same input matrix.; `question_6_convolution_filters/demo_convolution_filters.py:110-122`: Showing the kernel values directly makes it easier to justify why a filter responds strongly to one orientation and weakly to the others.; `question_6_convolution_filters/demo_convolution_filters.py:124-129`: Record the same kernel and response information in a structured report.; `question_6_convolution_filters/demo_convolution_filters.py:131`: Save the same information that was printed to the terminal.; `question_6_convolution_filters/demo_convolution_filters.py:133`: Create the output folder before writing the JSON file.; `question_6_convolution_filters/demo_convolution_filters.py:135-138`: Write the kernels and summary stats as formatted JSON.

File: `question_6_convolution_filters/demo_convolution_filters_image.py`

Classification: Repo-owned real-image preprocessing, filtering, visualization, and reporting script. Evidence anchor: `question_6_convolution_filters/demo_convolution_filters_image.py:1-363`

Purpose: Runs the same filter bank on real images and writes human-viewable artifacts that expose each stage of the image pipeline.

- Upstream handoff: Uses Pillow for image IO, but the convolution and reporting logic stays in repo-owned code.

Top import evidence: `__future__::annotations, argparse, json, os, sys, pathlib::Path, PIL::Image, ImageOps, convolution_filter_library::apply_all_directional_filters, matrix_stats, normalize_matrix_abs_to_uint8`

Function-by-function walkthrough

build_parser (function; `question_6_convolution_filters/demo_convolution_filters_image.py:53-77`)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: `question_6_convolution_filters/demo_convolution_filters_image.py:54-77`: Single contiguous control block.

collect_image_paths (function; `question_6_convolution_filters/demo_convolution_filters_image.py:80-113`)

Category: environment or artifact collection. Summary: Resolve the set of input images to process.

Failure signals raised directly here: `FileNotFoundError, ValueError`

Line-block walk: `question_6_convolution_filters/demo_convolution_filters_image.py:83-86`: Stop immediately if the requested source path does not exist.; `question_6_convolution_filters/demo_convolution_filters_image.py:88-95`: Reuse the directory pipeline by returning a one-item list.; `question_6_convolution_filters/demo_convolution_filters_image.py:97-99`: Stop if the source is neither a file nor a directory.; `question_6_convolution_filters/demo_convolution_filters_image.py:101-113`: Collect only supported image files from the directory root.

resolve_output_stems (function; `question_6_convolution_filters/demo_convolution_filters_image.py:116-142`)

Category: path/config resolution. Summary: Build one unique output stem for each image path.

Line-block walk: `question_6_convolution_filters/demo_convolution_filters_image.py:119-122`: Keep output directory names unique when two files share the same stem.; `question_6_convolution_filters/demo_convolution_filters_image.py:124-126`: Start from the source file stem.; `question_6_convolution_filters/demo_convolution_filters_image.py:128-133`: Add the suffix text when another file already uses this stem.; `question_6_convolution_filters/demo_convolution_filters_image.py:135-137`: Keep incrementing until the stem becomes unique.; `question_6_convolution_filters/demo_convolution_filters_image.py:139-142`: Reserve the chosen stem and store it for this image path.

load_rgb_image (function; `question_6_convolution_filters/demo_convolution_filters_image.py:145-152`)

Category: loading/ingest. Summary: Load one image as RGB.

Failure signals raised directly here: `RuntimeError`

Line-block walk: `question_6_convolution_filters/demo_convolution_filters_image.py:149-152`: Convert every input to RGB so the later steps see one image format.

preprocess_image (function; question_6_convolution_filters/demo_convolution_filters_image.py:155-163)

Category: local helper logic. Summary: Convert one RGB image into a thresholded grayscale map.

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:158-159: Convert to grayscale before stretching contrast or thresholding.; question_6_convolution_filters/demo_convolution_filters_image.py:161-163: Convert the grayscale image to a binary edge map.

image_to_matrix (function; question_6_convolution_filters/demo_convolution_filters_image.py:166-176)

Category: local helper logic. Summary: Convert one grayscale image into the matrix representation.

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:169: Convert the grayscale image into the 2D matrix shape used by the filters.; question_6_convolution_filters/demo_convolution_filters_image.py:171: Read the image pixels as one flat byte buffer.; question_6_convolution_filters/demo_convolution_filters_image.py:173-176: Rebuild the flat pixel buffer into a 2D float matrix.

uint8_matrix_to_image (function; question_6_convolution_filters/demo_convolution_filters_image.py:179-195)

Category: local helper logic. Summary: Convert one uint8 matrix back into a grayscale image.

Failure signals raised directly here: ValueError

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:182-184: Reject empty matrices before allocating the output image.; question_6_convolution_filters/demo_convolution_filters_image.py:186-190: Verify that every row has the same width.; question_6_convolution_filters/demo_convolution_filters_image.py:192: Flatten the matrix back into one grayscale image buffer.; question_6_convolution_filters/demo_convolution_filters_image.py:194-195: Copy every matrix value into the image buffer in row-major order.

create_feature_map_grid (function; question_6_convolution_filters/demo_convolution_filters_image.py:198-211)

Category: local helper logic. Summary: Arrange the original image and response maps into one grid.

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:201-204: Place the original image and derived maps into one contact sheet.; question_6_convolution_filters/demo_convolution_filters_image.py:206-207: Convert the linear position into one row and one column.; question_6_convolution_filters/demo_convolution_filters_image.py:209-211: Paste the current tile image into its grid cell.

write_json (function; question_6_convolution_filters/demo_convolution_filters_image.py:214-219)

Category: serialization/output writing. Summary: Write one JSON file to disk.

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:217: Create the destination folder before writing the JSON file.; question_6_convolution_filters/demo_convolution_filters_image.py:219: Write the JSON payload with indentation for readability.

format_cli_path (function; question_6_convolution_filters/demo_convolution_filters_image.py:222-229)

Category: summary/report formatting. Summary: Format a path for CLI output.

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:226-227: Prefer a path relative to the current shell directory.; question_6_convolution_filters/demo_convolution_filters_image.py:229: Fall back to the absolute path when no relative form exists.

portable_repo_path (function; question_6_convolution_filters/demo_convolution_filters_image.py:232-235)

Category: local helper logic. Summary: Return one repo-relative path string.

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:235: Store repo-relative paths so saved summaries remain portable.

process_image (function; question_6_convolution_filters/demo_convolution_filters_image.py:239-310)

Category: local helper logic. Summary: Owns the end-to-end real-image pipeline: load, preprocess, convolve, normalize, save each feature map, build the contact sheet, and serialize summary JSON.

Line-block walk: question_6_convolution_filters/demo_convolution_filters_image.py:242-245: Load the current input image and build the thresholded filter input.; question_6_convolution_filters/demo_convolution_filters_image.py:247-249: Apply every directional filter to the preprocessed image matrix.; question_6_convolution_filters/demo_convolution_filters_image.py:251-253: Create the per-image output folder before writing any artifacts.; question_6_convolution_filters/demo_convolution_filters_image.py:255-256: Save the thresholded grayscale image before the derived feature maps.; question_6_convolution_filters/demo_convolution_filters_image.py:258-264: Track the saved file paths and in-memory tiles for the grid image.; question_6_convolution_filters/demo_convolution_filters_image.py:266-268: Convert the signed response map into a viewable 8-bit image.; question_6_convolution_filters/demo_convolution_filters_image.py:270-274: Save the current filter response image.; question_6_convolution_filters/demo_convolution_filters_image.py:276: Combine the original image and all derived maps into one grid image.

main (function; question_6_convolution_filters/demo_convolution_filters_image.py:313-359)

Category: entrypoint orchestration. Summary: Run the image-processing CLI flow.

Line-block walk: `question_6_convolution_filters/demo_convolution_filters_image.py:319-321`: Resolve the source path and output root before processing any images.; `question_6_convolution_filters/demo_convolution_filters_image.py:323-334`: Collect the input images and reserve one output stem for each one.; `question_6_convolution_filters/demo_convolution_filters_image.py:336-339`: Group every artifact for this image under one output directory.; `question_6_convolution_filters/demo_convolution_filters_image.py:341-343`: Process the current image and record its summary paths.; `question_6_convolution_filters/demo_convolution_filters_image.py:345-356`: Report the summary path directly in single-image mode.; `question_6_convolution_filters/demo_convolution_filters_image.py:358-359`: Convert common validation failures into one CLI error line.

Not found in repo

- A citation naming the exact external source for the directional kernel design is not found in repo; the implementation is present, but an academic attribution note is not.
- A quantitative comparison against Sobel, Prewitt, or learned filters is not found in repo.

Q7 TLib VOC-to-Clipart Transfer Learning

Tracked scope: 4 top-level files, 2516 lines, 54 Python definitions, 3 shell functions.

What it is

- A repo-owned controller for TLib's VOC-to-Clipart domain adaptation example that adds environment doctoring, dataset prep, smoke subsets, stage skipping, profile defaults, compatibility repairs, and summary export.
- This folder is not inventing the transfer-learning algorithm; it is engineering a reproducible and inspectable path around TLib and Detectron2.

Why this folder exists in the repo

- To make a fragile multi-stage domain-adaptation workflow runnable on OSC without manual patching and hand-managed output bookkeeping.
- To expose the pipeline as understandable stages instead of one opaque external command.

Architecture and data flow

- Setup builds or repairs the TLib clone and rewrites a handful of upstream files for compatibility with newer torchvision, optional Detectron2 installs, and dataset metadata issues.
- The driver script resolves repo paths, profile defaults, interpreter, config files, output directories, environment summary, and device selection before any stage runs.
- Dataset preparation ensures full VOC2007/VOC2012/Clipart layouts exist and optionally derives reduced smoke subsets without mutating the full datasets.
- Stage execution is explicit: source-only training produces the baseline checkpoint, D-adapt runs sequential phases that consume the previous checkpoint, visualization renders saved predictions, and report/full-pipeline summarize outputs into JSON and Markdown.
- The wrapper's value is the controller logic: skip if outputs exist, remove only stage-specific artifacts on `--force`, and keep summaries portable with repo-relative paths.

Tracked files and responsibilities

`question_7_transfer_learning/README.md` -> Repo-owned documentation/evidence file. (219 lines).

`question_7_transfer_learning/demo_tlib_object_detection.py` -> Repo-owned high-level TLib pipeline wrapper and reporting driver. (1741 lines).

`question_7_transfer_learning/run_tlib_osc.sh` -> Repo-owned execution wrapper that repairs, doctors, and runs the Q7 pipeline. (96 lines).

`question_7_transfer_learning/setup_tlib_osc.sh` -> Repo-owned setup and compatibility-patch shell script. (460 lines).

Data transformations and artifact lifecycle

- Artifact counts now present on disk: outputs 4, runs 0, datasets 63490, logs 44.
- Tracked smoke summary exists at `question_7_transfer_learning/outputs/voc2clipart_smoke/summary.md` with 2 stage sections.
- The tracked smoke summary currently shows zero AP metrics, which is useful evidence that the reporting layer works even when results are weak.
- Tracked visualization PNGs present: 8.

Sample tracked outputs: `question_7_transfer_learning/outputs/voc2clipart_smoke`,
`question_7_transfer_learning/outputs/voc2clipart_smoke/summary.json`,
`question_7_transfer_learning/outputs/voc2clipart_smoke/summary.md`,
`question_7_transfer_learning/outputs/voc2clipart_smoke 2`

Sample tracked datasets: `question_7_transfer_learning/datasets/VOC2007`,
`question_7_transfer_learning/datasets/VOC2007/Annotations`,
`question_7_transfer_learning/datasets/VOC2007/Annotations/000001.xml`,
`question_7_transfer_learning/datasets/VOC2007/Annotations/000002.xml`,
`question_7_transfer_learning/datasets/VOC2007/Annotations/000003.xml`,
`question_7_transfer_learning/datasets/VOC2007/Annotations/000004.xml`

Sample tracked logs: `question_7_transfer_learning/logs/d_adapt_smoke`,
`question_7_transfer_learning/logs/d_adapt_smoke/faster_rcnn_R_101_C4`,
`question_7_transfer_learning/logs/d_adapt_smoke/faster_rcnn_R_101_C4/voc2clipart`,
`question_7_transfer_learning/logs/d_adapt_smoke/faster_rcnn_R_101_C4/voc2clipart/phase1`,

question_7_transfer_learning/logs/d_adapt_smoke/faster_rcnn_R_101_C4/voc2clipart/phase1/adaptor_status.json,
question_7_transfer_learning/logs/d_adapt_smoke/faster_rcnn_R_101_C4/voc2clipart/phase1/bbox

Original repo logic vs upstream backbone logic

- Repo-owned logic here is mostly the tracked `.py/.sh` wrapper layer plus any artifact shaping, environment enforcement, and summary-writing logic called out below.
- Builds commands for `source_only.py`, `d_adapt.py`, and `visualize.py` under `external/Transfer-Learning-Library/examples/domain_adaptation/object_detection`.
- The actual detector training/adaptation/visualization is therefore TLLib/Detectron2 code; the repo-owned layer is the controller around it.
- Delegates the heavy lifting to `demo_tllib_object_detection.py`, which in turn delegates stage execution to TLLib's upstream scripts.
- Clones `thuml/Transfer-Learning-Library` and rewrites specific files under `tllib/...` to keep that checkout compatible with the validated OSC stack.

What to explain deeply vs what to mention briefly

- Explain deeply: Explain `apply_profile_defaults(...)`, `resolve_dataset_paths(...)`, `build_source_only_command(...)`, and `build_dadapt_command(...)` because they translate the repo's intent into the exact upstream TLLib invocation.
- Explain deeply: Explain `create_smoke_subset(...)` as a deterministic data-reduction strategy that keeps the dataset contract intact.
- Explain deeply: Explain `maybe_run_stage(...)`, `summarize_stage(...)`, and `write_summary(...)` because they are where the wrapper becomes resumable and reviewable instead of merely executable.
- Explain deeply: Explain the compatibility patcher in `setup_tllib_osc.sh`; it is one of the clearest examples of repo-authored engineering around an upstream backbone.
- Mention briefly: Mention Detectron2 and TLLib model internals briefly unless you have separately studied the upstream implementation in `external/Transfer-Learning-Library`.
- Mention briefly: Mention command-printing and dry-run helpers briefly; they are useful but not the conceptual center.

Minimal run path

- Setup: `bash question_7_transfer_learning/setup_tllib_osc.sh`
- Doctor: `python3.9 question_7_transfer_learning/demo_tllib_object_detection.py --mode doctor`
- Smoke pipeline: `ALLOW_CPU=1 PROFILE=smoke bash question_7_transfer_learning/run_tllib_osc.sh`
- Benchmark pipeline: `bash osc_gpu_batch.sh --account <OSC_ACCOUNT> --time 04:00:00 -- env PROFILE=benchmark bash question_7_transfer_learning/run_tllib_osc.sh`

Per-file walkthrough

File: `question_7_transfer_learning/README.md`

Classification: Repo-owned documentation/evidence file. Evidence anchor: `question_7_transfer_learning/README.md:1-219`

Purpose: Acts as explicit repo evidence for workflow intent, minimal run steps, and ownership boundaries.

Key headings present in the tracked README: `Transfer Learning Object Detection with TLLib`, `Overview`, `Setup`, `Self-Healing Behavior`, `Doctor Check`, `Profiles`, `Smoke Profile`, `Benchmark Profile`, `Direct Wrapper Usage`, `Outputs`

File: `question_7_transfer_learning/demo_tllib_object_detection.py`

Classification: Repo-owned high-level TLLib pipeline wrapper and reporting driver. Evidence anchor: `question_7_transfer_learning/demo_tllib_object_detection.py:1-1741`

Purpose: Normalizes configuration, validates the environment, prepares datasets, builds TLLib commands, controls stage-skipping, and writes portable summaries.

- Upstream handoff: Builds commands for `source_only.py`, `d_adapt.py`, and `visualize.py` under `external/Transfer-Learning-Library/examples/domain_adaptation/object_detection`.
- Upstream handoff: The actual detector training/adaptation/visualization is therefore TLLib/Detectron2 code; the repo-owned layer is the controller around it.

Top import evidence: `__future__::annotations`, `argparse`, `ast`, `json`, `os`, `platform`, `re`, `shlex`

Function-by-function walkthrough

resolve_path (function; question_7_transfer_learning/demo_tllib_object_detection.py:123-133)

Category: path/config resolution. Summary: Resolve a path against the given base directory.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:126-127: Keep optional CLI values optional.; question_7_transfer_learning/demo_tllib_object_detection.py:129: Expand "~" before checking whether the path is already absolute.; question_7_transfer_learning/demo_tllib_object_detection.py:131-133: Anchor relative paths to the requested base directory.

portable_repo_path (function; question_7_transfer_learning/demo_tllib_object_detection.py:136-141)

Category: local helper logic. Summary: Return a repository-relative version of one path.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:141: Keep tracked summaries portable and avoid leaking machine-specific absolute filesystem paths into committed artifacts. Convert the resolved path back into a repo-relative string.

resolve_python (function; question_7_transfer_learning/demo_tllib_object_detection.py:144-154)

Category: path/config resolution. Summary: Resolve the Python executable to use.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:147-148: Honor an explicit interpreter override first.; question_7_transfer_learning/demo_tllib_object_detection.py:150-152: Prefer the repository-local virtual environment when it exists.; question_7_transfer_learning/demo_tllib_object_detection.py:154: Fall back to the interpreter running this file.

resolve_output_dir (function; question_7_transfer_learning/demo_tllib_object_detection.py:157-162)

Category: path/config resolution. Summary: Resolve the output directory path.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:160-162: Output directory defaults are guaranteed upstream, so unwrap here.

apply_profile_defaults (function; question_7_transfer_learning/demo_tllib_object_detection.py:165-173)

Category: local helper logic. Summary: Makes smoke and benchmark real operational modes instead of mere labels by filling in missing runtime knobs from repo-owned defaults.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:169: Most runs need only the profile selection, so populate the remaining stage settings from the chosen profile defaults.; question_7_transfer_learning/demo_tllib_object_detection.py:171-173: Only fill fields that remain unset after CLI parsing.

find_object_detection_dir (function; question_7_transfer_learning/demo_tllib_object_detection.py:176-192)

Category: local helper logic. Summary: Find TLib's object-detection example directory.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:180: Prefer the canonical TLib location, then fall back to a search so the script still works if the upstream repository layout changes slightly.; question_7_transfer_learning/demo_tllib_object_detection.py:182-184: Return the standard location immediately when it matches.; question_7_transfer_learning/demo_tllib_object_detection.py:186-192: Fall back to a repo-wide search for the source_only entrypoint.

find_dadapt_dir (function; question_7_transfer_learning/demo_tllib_object_detection.py:195-206)

Category: local helper logic. Summary: Find TLib's D-adapt directory.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:198-199: Some TLib checkouts use d_adapt/ while others have d-adapt/ naming, so we probe both instead of assuming one exact folder name.; question_7_transfer_learning/demo_tllib_object_detection.py:201-206: Return the first folder that contains the expected script.

find_missing_modules (function; question_7_transfer_learning/demo_tllib_object_detection.py:209-233)

Category: subprocess orchestration. Summary: Check which Python modules are missing.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:212-228: Ask the target interpreter to report any missing imports in one pass.; question_7_transfer_learning/demo_tllib_object_detection.py:230-231: Exit code 0 means every requested module imported successfully.; question_7_transfer_learning/demo_tllib_object_detection.py:233: Otherwise return the missing module names printed by the probe.

run_python_probe (function; question_7_transfer_learning/demo_tllib_object_detection.py:236-246)

Category: stage execution. Summary: Run a small Python probe and return its output.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:239-244: Run one short inline Python snippet under the chosen interpreter.; question_7_transfer_learning/demo_tllib_object_detection.py:246: Strip the trailing newline so the value can be embedded directly.

ensure_supported_runtime_python (function; question_7_transfer_learning/demo_tllib_object_detection.py:249-268)

Category: validation or readiness gate. Summary: Reject unsupported target interpreters before TLLib preflight runs.

Failure signals raised directly here: RuntimeError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:250-268: Single contiguous control block.

runtime_cuda_available (function; question_7_transfer_learning/demo_tllib_object_detection.py:271-284)

Category: subprocess orchestration. Summary: Return True when the target runtime reports CUDA availability.

osc_gpu_batch_example (function; question_7_transfer_learning/demo_tllib_object_detection.py:287-293)

Category: local helper logic. Summary: Build one OSC GPU launcher example command.

validate_python_environment (function; question_7_transfer_learning/demo_tllib_object_detection.py:296-315)

Category: validation or contract enforcement. Summary: Check that the required Python packages are installed.

Failure signals raised directly here: RuntimeError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:304-315: Stop before launching any long training job if the Detectron2 stack is not importable; that is the most common fresh-environment failure mode.

probe_tllib_object_detection_import (function; question_7_transfer_learning/demo_tllib_object_detection.py:318-347)

Category: subprocess orchestration. Summary: Probe whether the TLLib object-detection module imports cleanly.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:319-347: Single contiguous control block.

ensure_tllib_object_detection_importable (function; question_7_transfer_learning/demo_tllib_object_detection.py:350-367)

Category: validation or readiness gate. Summary: Fail early when TLLib's object-detection stack is still broken.

Failure signals raised directly here: RuntimeError

dataset_layout_ok (function; question_7_transfer_learning/demo_tllib_object_detection.py:370-373)

Category: local helper logic. Summary: Return True when the dataset layout looks complete.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:373: All required VOC-style subpaths must exist together.

validate_dataset_dir (function; question_7_transfer_learning/demo_tllib_object_detection.py:376-390)

Category: validation or contract enforcement. Summary: Check that one dataset directory is complete.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:379-383: Stop early if the dataset root itself is missing.; question_7_transfer_learning/demo_tllib_object_detection.py:385-390: Report the exact missing pieces when the root exists but the layout is incomplete.

validate_source_only_inputs (function; question_7_transfer_learning/demo_tllib_object_detection.py:393-407)

Category: validation or contract enforcement. Summary: Check the source-only inputs before running TLLib.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:396-400: The detector config file must exist before any TLLib script can start.; question_7_transfer_learning/demo_tllib_object_detection.py:402-404: Validate each dataset root individually so the error names the missing dataset.; question_7_transfer_learning/demo_tllib_object_detection.py:406-407: Confirm the upstream source-only entripoint exists where expected.

validate_dadapt_inputs (function; question_7_transfer_learning/demo_tllib_object_detection.py:410-422)

Category: validation or contract enforcement. Summary: Check the D-adapt inputs before running TLLib.

Failure signals raised directly here: FileNotFoundError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:413: Reuse the baseline validation first because D-adapt needs the same datasets and config.; question_7_transfer_learning/demo_tllib_object_detection.py:415-416: Confirm the D-adapt script exists in the chosen folder.; question_7_transfer_learning/demo_tllib_object_detection.py:418-422: Later phases cannot start unless the input checkpoint already exists.

extract_archive (function; question_7_transfer_learning/demo_tllib_object_detection.py:427-439)

Category: structured extraction from upstream outputs. Summary: Extract one dataset archive.

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:432`: Clipart ships as ZIP while VOC ships as tar; this helper hides that packaging difference so dataset preparation becomes one code path.;
`question_7_transfer_learning/demo_tllib_object_detection.py:434-436`: Extract ZIP archives directly into the destination root.;
`question_7_transfer_learning/demo_tllib_object_detection.py:438-439`: Otherwise treat the archive as a tarball.

download_dataset (function; `question_7_transfer_learning/demo_tllib_object_detection.py:442-460`)

Category: download or remote fetch. Summary: Download one dataset archive when needed.

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:450-460`: Keep the downloaded archives next to the extracted folders so repeated runs can reuse them without re-downloading large files.

write_split_file (function; `question_7_transfer_learning/demo_tllib_object_detection.py:463-468`)

Category: serialization/output writing. Summary: Write one image-id split file.

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:466`: Create the split directory before writing the file.;
`question_7_transfer_learning/demo_tllib_object_detection.py:468`: Preserve the trailing newline format expected by many dataset tools.

ensure_trainval_split (function; `question_7_transfer_learning/demo_tllib_object_detection.py:471-507`)

Category: validation or readiness gate. Summary: Make sure the trainval split file exists.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:476-478`: Leave an existing trainval split untouched.;
`question_7_transfer_learning/demo_tllib_object_detection.py:481-484`: Clipart sometimes lacks trainval.txt, but TLLib expects it. Rebuild it by merging train/test IDs so the dataset matches the script's assumptions.;
`question_7_transfer_learning/demo_tllib_object_detection.py:486-487`: Read the train ids when that split exists.;
`question_7_transfer_learning/demo_tllib_object_detection.py:489-494`: Read the test ids when that split exists.;
`question_7_transfer_learning/demo_tllib_object_detection.py:496-507`: Keep only the first occurrence of each id.

read_split_file (function; `question_7_transfer_learning/demo_tllib_object_detection.py:510-521`)

Category: local helper logic. Summary: Read one split file into a list of image ids.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:513`: Build the expected split-file path from the dataset root and split name.;
`question_7_transfer_learning/demo_tllib_object_detection.py:515-516`: Synthesize trainval.txt on demand when that is the only missing file.;
`question_7_transfer_learning/demo_tllib_object_detection.py:518-519`: Stop here if the split file is still missing.;
`question_7_transfer_learning/demo_tllib_object_detection.py:521`: Return one stripped image id per non-empty line.

copy_subset_files (function; `question_7_transfer_learning/demo_tllib_object_detection.py:524-548`)

Category: artifact copying or dataset materialization. Summary: Copy the selected subset files into the smoke dataset.

Failure signals raised directly here: `FileNotFoundError`

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:532-533`: Copy only the files referenced by the chosen split IDs so the smoke subset preserves the VOC-style directory layout while staying tiny.;
`question_7_transfer_learning/demo_tllib_object_detection.py:535-541`: Start from the split id stem and try the allowed file extensions.;
`question_7_transfer_learning/demo_tllib_object_detection.py:543-546`: Fail if no annotation or image file matches the requested id.;
`question_7_transfer_learning/demo_tllib_object_detection.py:548`: Copy the resolved file into the mirrored subset folder.

create_smoke_subset (function; `question_7_transfer_learning/demo_tllib_object_detection.py:551-587`)

Category: local helper logic. Summary: Creates deterministic reduced datasets without breaking the VOC-style directory contract, which is central to explaining why smoke mode still exercises the real pipeline shape.

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:554-564`: Reuse an existing subset only when every tracked split still has the right size.;
`question_7_transfer_learning/demo_tllib_object_detection.py:567-568`: Preserve per-split semantics while truncating each split to a tiny prefix. That gives a deterministic local test set without inventing new metadata.;
`question_7_transfer_learning/demo_tllib_object_detection.py:570-574`: Keep the first N ids from each split to make smoke mode deterministic.;
`question_7_transfer_learning/demo_tllib_object_detection.py:576-577`: Copy the union once so shared ids are not duplicated.;
`question_7_transfer_learning/demo_tllib_object_detection.py:579-583`: Replace any stale subset tree before copying the new files.;
`question_7_transfer_learning/demo_tllib_object_detection.py:585-587`: Rewrite each split file so it matches the reduced subset exactly.

build_smoke_detector_overrides (function; `question_7_transfer_learning/demo_tllib_object_detection.py:590-623`)

Category: configuration or artifact construction. Summary: Build the detector overrides for smoke mode.

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:593-595`: Benchmark mode leaves the upstream config untouched.;
`question_7_transfer_learning/demo_tllib_object_detection.py:598-623`: Smoke mode aggressively shrinks the Detectron2 workload so the run can finish quickly without editing TLLib's upstream config files.

ensure_full_datasets (function; question_7_transfer_learning/demo_tllib_object_detection.py:626-643)

Category: validation or readiness gate. Summary: Make sure the full datasets are available.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:630-631: Reuse complete datasets when they are already on disk.; question_7_transfer_learning/demo_tllib_object_detection.py:633-634: Normalize Clipart even when the raw files already exist.; question_7_transfer_learning/demo_tllib_object_detection.py:636-638: Without download permission, surface the validation error now.; question_7_transfer_learning/demo_tllib_object_detection.py:640-641: Download and extract the missing dataset from its known source.; question_7_transfer_learning/demo_tllib_object_detection.py:643: Clipart still needs the synthesized trainval file after extraction.

ensure_smoke_subsets (function; question_7_transfer_learning/demo_tllib_object_detection.py:646-664)

Category: validation or readiness gate. Summary: Make sure the smoke subsets are available.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:649-664: Each dataset gets its own reduced clone so smoke mode can point Tllib at a separate root instead of mutating or trimming the full datasets.

parse_adapt_confidence_ratios (function; question_7_transfer_learning/demo_tllib_object_detection.py:667-675)

Category: argument or text parsing. Summary: Parse the D-adapt confidence ratios.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:670: Later D-adapt phases can tighten pseudo-label confidence thresholds. Comma-separated CLI input keeps that schedule easy to explain and tweak.; question_7_transfer_learning/demo_tllib_object_detection.py:672-673: Use the built-in defaults when the CLI value is blank.; question_7_transfer_learning/demo_tllib_object_detection.py:675: Otherwise parse one floating-point ratio per comma-separated token.

build_solver_steps_value (function; question_7_transfer_learning/demo_tllib_object_detection.py:678-695)

Category: configuration or artifact construction. Summary: Build the solver step override string.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:681: When we shorten training for smoke mode, the LR step schedule also needs to shrink; otherwise Detectron2 would step outside the tiny run window.; question_7_transfer_learning/demo_tllib_object_detection.py:683-685: A one-iteration run cannot contain any solver steps.; question_7_transfer_learning/demo_tllib_object_detection.py:687-688: Place one step around halfway and one later in the run.; question_7_transfer_learning/demo_tllib_object_detection.py:690-695: Keep only unique steps that still fall before the last iteration.

append_option (function; question_7_transfer_learning/demo_tllib_object_detection.py:700-706)

Category: local helper logic. Summary: Append one CLI option when it has a value.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:703-704: Skip optional CLI flags that were left unset.; question_7_transfer_learning/demo_tllib_object_detection.py:706: Append the flag and its value as separate argv entries.

append_cfg_override (function; question_7_transfer_learning/demo_tllib_object_detection.py:709-715)

Category: local helper logic. Summary: Append one config override pair.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:712-713: Skip unset config overrides.; question_7_transfer_learning/demo_tllib_object_detection.py:715: Detectron2-style overrides are passed as alternating key/value pairs.

build_source_only_command (function; question_7_transfer_learning/demo_tllib_object_detection.py:718-757)

Category: upstream command construction. Summary: Translates the repo's stage concept into the exact Tllib CLI for baseline training and evaluation.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:721-741: This stage trains the baseline detector on VOC and evaluates it on both VOC2007 test and the target-domain Clipart set.; question_7_transfer_learning/demo_tllib_object_detection.py:743-745: Override the detector schedule when the selected profile asked for it.; question_7_transfer_learning/demo_tllib_object_detection.py:747-751: Apply the common runtime/output overrides after the base dataset arguments.; question_7_transfer_learning/demo_tllib_object_detection.py:753: Add the smoke-only resource reductions when the smoke profile is active.; question_7_transfer_learning/demo_tllib_object_detection.py:755-757: Pass through resume so Detectron2 can continue an interrupted run.

build_dadapt_command (function; question_7_transfer_learning/demo_tllib_object_detection.py:760-820)

Category: upstream command construction. Summary: Builds one sequential adaptation phase, injecting both Tllib adaptor knobs and Detectron2 config overrides.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:763-782: D-adapt keeps the same detector backbone/config but adds Tllib's category and bbox adaptor training around a starting checkpoint.; question_7_transfer_learning/demo_tllib_object_detection.py:784-798: Append the category-adaptor and bbox-adaptor tuning knobs.; question_7_transfer_learning/demo_tllib_object_detection.py:800-802: Apply the detector-side schedule overrides for the adaptation phase.; question_7_transfer_learning/demo_tllib_object_detection.py:804-810: Point the phase at its output folder, seed, and starting weights.; question_7_transfer_learning/demo_tllib_object_detection.py:812-813: Reuse the smoke detector reductions here too when smoke mode is active.; question_7_transfer_learning/demo_tllib_object_detection.py:816: Phase 1 starts from the baseline model directly; later phases can optionally refine pseudo-label confidence with a stricter ratio.; question_7_transfer_learning/demo_tllib_object_detection.py:818-820: Pass resume through to the Tllib script when requested.

build_visualize_command (function; question_7_transfer_learning/demo_tllib_object_detection.py:823-849)

Category: upstream command construction. Summary: Build the visualization command.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:825-846: Reuse Tllib's visualization script to render predictions from one checkpoint.; question_7_transfer_learning/demo_tllib_object_detection.py:848-849: Keep visualization resource usage aligned with the chosen profile.

resolve_visualize_config (function; question_7_transfer_learning/demo_tllib_object_detection.py:852-857)

Category: path/config resolution. Summary: Pick the config file for visualization.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:855: D-adapt checkpoints belong with the D-adapt config; the source-only baseline keeps the original detector config.; question_7_transfer_learning/demo_tllib_object_detection.py:857: Baseline checkpoints use the source-only config.

resolve_expected_stage_path (function; question_7_transfer_learning/demo_tllib_object_detection.py:860-863)

Category: path/config resolution. Summary: Resolve the stage path to clean or check.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:863: Clean/check the parent directory when the expected artifact is a file.

print_command (function; question_7_transfer_learning/demo_tllib_object_detection.py:867-872)

Category: local helper logic. Summary: Print the command before running it.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:870: Print the working directory first so relative script paths are clear.; question_7_transfer_learning/demo_tllib_object_detection.py:872: Quote each argv item the same way a shell would display it.

run_command (function; question_7_transfer_learning/demo_tllib_object_detection.py:875-883)

Category: stage execution. Summary: Run the command unless this is a dry run.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:878: Always print the command so the execution plan is visible.; question_7_transfer_learning/demo_tllib_object_detection.py:880-881: Stop after printing when dry-run mode is active.; question_7_transfer_learning/demo_tllib_object_detection.py:883: Execute the upstream Tllib command in its expected working directory.

maybe_run_stage (function; question_7_transfer_learning/demo_tllib_object_detection.py:886-908)

Category: stage execution. Summary: This is the resumability contract: skip if the expected output exists, or surgically clean only the current stage when --force is set.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:889-898: Treat a completed checkpoint directory as the contract for that stage. This keeps the pipeline resumable across repeated runs.; question_7_transfer_learning/demo_tllib_object_detection.py:901-905: force removes only the specific stage artifact directory/file so the rerun starts cleanly without touching unrelated outputs.; question_7_transfer_learning/demo_tllib_object_detection.py:907-908: Print the stage label right before launching its command.

parse_eval_metrics_from_log (function; question_7_transfer_learning/demo_tllib_object_detection.py:911-951)

Category: argument or text parsing. Summary: Parse the latest evaluation metrics from a log file.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:914-916: Without a log file there are no metrics to summarize.; question_7_transfer_learning/demo_tllib_object_detection.py:919-921: Tllib/Detectron2 logs metrics in more than one textual format depending on the code path, so the parser accepts either the dict payload or the table.; question_7_transfer_learning/demo_tllib_object_detection.py:923-924: First look for the inline Python-dict metric payload.; question_7_transfer_learning/demo_tllib_object_detection.py:926-936: If no dict payload exists on this line, try the table row format instead.; question_7_transfer_learning/demo_tllib_object_detection.py:938-943: Normalize the logged dict string so ast.literal_eval can parse it.; question_7_transfer_learning/demo_tllib_object_detection.py:945-946: Keep the latest bbox metrics block found in the log.; question_7_transfer_learning/demo_tllib_object_detection.py:948-949: Prefer the explicit bbox dict when it exists.; question_7_transfer_learning/demo_tllib_object_detection.py:951: Otherwise fall back to whatever metrics were parsed from the table.

summarize_stage (function; question_7_transfer_learning/demo_tllib_object_detection.py:954-978)

Category: summary/report formatting. Summary: Extracts the minimum facts that make the pipeline defensible after the fact: checkpoints, log existence, metrics, adaptor status, and visualization counts.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:957-959: Resolve the common stage artifacts from the output directory.; question_7_transfer_learning/demo_tllib_object_detection.py:961-970: Collect the files and metrics that describe one completed stage.; question_7_transfer_learning/demo_tllib_object_detection.py:972-973: Include adaptor health/status when the phase wrote it.; question_7_transfer_learning/demo_tllib_object_detection.py:975-978: Count any saved PNG visualizations tied to this stage.

write_summary (function; question_7_transfer_learning/demo_tllib_object_detection.py:981-1026)

Category: serialization/output writing. Summary: Writes both machine-readable and human-readable reports so the pipeline leaves a portable audit trail.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:984-986: Make sure the summary folder exists before writing either file.; question_7_transfer_learning/demo_tllib_object_detection.py:988-989: Write the machine-readable summary first.; question_7_transfer_learning/demo_tllib_object_detection.py:992-1002: The Markdown file is the readable handoff, while the JSON

preserves the same facts in a machine-friendly form.; question_7_transfer_learning/demo_tllib_object_detection.py:1004-1023: Start one markdown section per pipeline stage.; question_7_transfer_learning/demo_tllib_object_detection.py:1025-1026: Write the human-readable markdown companion file.

collect_environment_summary (function;
question_7_transfer_learning/demo_tllib_object_detection.py:1029-1063)

Category: environment or artifact collection. Summary: Collect the environment details for the summary.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:1032-1037: Start with the host operating system and architecture.; question_7_transfer_learning/demo_tllib_object_detection.py:1039-1047: Ask the target interpreter for its Python version.; question_7_transfer_learning/demo_tllib_object_detection.py:1049-1057: Query each package version from the target interpreter directly.; question_7_transfer_learning/demo_tllib_object_detection.py:1059-1063: Add one note when the host is a slower Apple Silicon setup.

resolve_model_device (function; question_7_transfer_learning/demo_tllib_object_detection.py:1066-1086)

Category: path/config resolution. Summary: Pick the runtime device to use.

Failure signals raised directly here: RuntimeError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:1069: 'auto' means "ask the runtime" rather than guessing from the host. If torch is missing or fails to import, fall back to CPU deterministically.; question_7_transfer_learning/demo_tllib_object_detection.py:1071-1082: Respect an explicit device choice first.; question_7_transfer_learning/demo_tllib_object_detection.py:1084-1086: Without torch installed, the safest fallback is CPU.

build_dataset_paths_from_root (function;
question_7_transfer_learning/demo_tllib_object_detection.py:1089-1096)

Category: configuration or artifact construction. Summary: Build the dataset paths from one root folder.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:1092-1096: Keep the three Tllib dataset roots grouped under one shared root.

resolve_dataset_paths (function; question_7_transfer_learning/demo_tllib_object_detection.py:1099-1123)

Category: path/config resolution. Summary: Resolve the active dataset paths.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:1102-1105: Resolve the full-dataset root and the smoke-dataset root first.; question_7_transfer_learning/demo_tllib_object_detection.py:1107-1109: Expand each root into the three dataset-specific directories.; question_7_transfer_learning/demo_tllib_object_detection.py:1112-1119: The active dataset root switches with the profile, but explicit CLI paths still win so the script can target custom dataset locations when needed.; question_7_transfer_learning/demo_tllib_object_detection.py:1121-1123: Let explicit dataset-path overrides replace the profile-derived defaults.

resolve_config_file (function; question_7_transfer_learning/demo_tllib_object_detection.py:1126-1144)

Category: path/config resolution. Summary: Resolve the config file path.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:1129-1130: Accept either an absolute path or the same relative path anchored under Tllib's source-only and D-adapt folders.; question_7_transfer_learning/demo_tllib_object_detection.py:1132-1133: Absolute config paths pass straight through.; question_7_transfer_learning/demo_tllib_object_detection.py:1135-1138: Try the primary script directory first.; question_7_transfer_learning/demo_tllib_object_detection.py:1140-1142: Fall back to the secondary directory when provided.; question_7_transfer_learning/demo_tllib_object_detection.py:1144: Return the primary candidate even when it does not exist so validation can report it.

determine_latest_phase (function; question_7_transfer_learning/demo_tllib_object_detection.py:1147-1154)

Category: local helper logic. Summary: Find the latest completed adapt phase.

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:1152-1154: Track the highest phase index whose final checkpoint exists.

resolve_stage_weights (function; question_7_transfer_learning/demo_tllib_object_detection.py:1157-1176)

Category: path/config resolution. Summary: Resolve which checkpoint weights to use for a stage.

Failure signals raised directly here: FileNotFoundError, ValueError

Line-block walk: question_7_transfer_learning/demo_tllib_object_detection.py:1160-1161: Visualization/reporting can target the baseline or the latest completed adaptation phase, so centralize that checkpoint selection logic here.; question_7_transfer_learning/demo_tllib_object_detection.py:1163-1174: The baseline stage always reads from the source-only output folder.; question_7_transfer_learning/demo_tllib_object_detection.py:1176: phaseN values map directly to phaseN/model_final.pth.

print_doctor_summary (function; question_7_transfer_learning/demo_tllib_object_detection.py:1181-1215)

Category: local helper logic. Summary: Print the environment and dataset summary.

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:1183-1185`: doctor prints dependency and dataset readiness without running training.; `question_7_transfer_learning/demo_tllib_object_detection.py:1187-1204`: Print one normalized environment field per line.; `question_7_transfer_learning/demo_tllib_object_detection.py:1206-1210`: Show whether each full dataset root is ready right now.; `question_7_transfer_learning/demo_tllib_object_detection.py:1212-1215`: Show the same readiness check for the smoke subsets.

build_parser (function; `question_7_transfer_learning/demo_tllib_object_detection.py:1218-1400`)

Category: configuration or artifact construction. Summary: Build the CLI parser.

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:1261-1295`: Core repo/config/data locations.; `question_7_transfer_learning/demo_tllib_object_detection.py:1297-1334`: Detector schedule overrides.; `question_7_transfer_learning/demo_tllib_object_detection.py:1336-1354`: Smoke-specific detector reductions so local runs finish quickly.; `question_7_transfer_learning/demo_tllib_object_detection.py:1356-1400`: Output locations for checkpoints, PNGs, and the final summary.

main (function; `question_7_transfer_learning/demo_tllib_object_detection.py:1403-1737`)

Category: entrypoint orchestration. Summary: Owns the full controller flow: resolve paths/profile/device, prepare datasets, validate the runtime, run stages, and materialize the final report.

Failure signals raised directly here: `FileNotFoundError`, `ValueError`

Line-block walk: `question_7_transfer_learning/demo_tllib_object_detection.py:1408`: Resolve all paths, profile defaults, and environment facts once so each mode branch below can focus only on its own stage logic.; `question_7_transfer_learning/demo_tllib_object_detection.py:1410-1413`: Resolve the repo root before looking for any upstream scripts.; `question_7_transfer_learning/demo_tllib_object_detection.py:1415-1429`: Resolve the interpreter, script directories, configs, and output folders.; `question_7_transfer_learning/demo_tllib_object_detection.py:1431`: Expand the confidence-ratio schedule so every later adapt phase has one value.; `question_7_transfer_learning/demo_tllib_object_detection.py:1434-1436`: If fewer ratios are provided than phases, repeat the last one so every post-phase1 stage still receives an explicit confidence setting.; `question_7_transfer_learning/demo_tllib_object_detection.py:1438-1442`: Resolve full/smoke dataset roots and inspect the runtime environment.; `question_7_transfer_learning/demo_tllib_object_detection.py:1444-1453`: doctor is the lightweight preflight: show environment status without mutating anything.; `question_7_transfer_learning/demo_tllib_object_detection.py:1455-1457`: Dataset preparation is split out so downloads and subsets can be reused across runs.

File: `question_7_transfer_learning/run_tllib_osc.sh`

Classification: Repo-owned execution wrapper that repairs, doctors, and runs the Q7 pipeline. Evidence anchor: `question_7_transfer_learning/run_tllib_osc.sh:1-96`

Purpose: Provides the practical end-to-end entrypoint that repairs the environment, runs doctor, resolves the device, and launches the selected full pipeline.

- Upstream handoff: Delegates the heavy lifting to `demo_tllib_object_detection.py`, which in turn delegates stage execution to TLib's upstream scripts.

Top-level shell stages

`question_7_transfer_learning/run_tllib_osc.sh:2-3`: `!/usr/bin/env bash`

`question_7_transfer_learning/run_tllib_osc.sh:5-4`: Run the repo-owned Q7 helper flow on OSC. The wrapper first repairs or

`question_7_transfer_learning/run_tllib_osc.sh:6-5`: bootstraps the TLib clone, then runs doctor, then launches the selected

`question_7_transfer_learning/run_tllib_osc.sh:7-18`: full pipeline profile with the resolved device configuration.

`question_7_transfer_learning/run_tllib_osc.sh:20-23`: A missing runtime means setup must build the environment from scratch.

`question_7_transfer_learning/run_tllib_osc.sh:25-24`: When the environment already exists, prefer the cheaper repair-only path so

`question_7_transfer_learning/run_tllib_osc.sh:26-36`: compatibility fixes are refreshed before doctor or training starts.

`question_7_transfer_learning/run_tllib_osc.sh:38-37`: Require a real OSC GPU allocation before benchmark-style execution.

`question_7_transfer_learning/run_tllib_osc.sh:39-43`: `shellcheck disable=SC1090`

`question_7_transfer_learning/run_tllib_osc.sh:45-51`: Require CUDA by default unless CPU execution was requested explicitly.

`question_7_transfer_learning/run_tllib_osc.sh:53-60`: Keep one resolved device string for the Python driver call below.

`question_7_transfer_learning/run_tllib_osc.sh:62-69`: Forward any extra CLI flags to the Python driver.

`question_7_transfer_learning/run_tllib_osc.sh:71-70`: Keep the explicit environment check in the shell transcript before the

`question_7_transfer_learning/run_tllib_osc.sh:72-71`: longer-running training stages begin.

`question_7_transfer_learning/run_tllib_osc.sh:73-79`: Run the environment check before starting the full pipeline.

`question_7_transfer_learning/run_tllib_osc.sh:81-96`: Run the full pipeline with the resolved profile and device.

Representative commands and side effects

`set -euo pipefail`

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
TLIB_REPO_DIR="${TLIB_REPO_DIR:-${REPO_ROOT}/external/Transfer-Learning-Library}"
PREFLIGHT_SCRIPT="${REPO_ROOT}/osc_gpu_preflight.sh"
SETUP_SCRIPT="${SCRIPT_DIR}/setup_tllib_osc.sh"
PYTHON_BIN="${PYTHON_BIN:-${TLIB_REPO_DIR}/.venv/bin/python}"
ALLOW_CPU="${ALLOW_CPU:-0}"
```

File: question_7_transfer_learning/setup_tllib_osc.sh

Classification: Repo-owned setup and compatibility-patch shell script. Evidence anchor: question_7_transfer_learning/setup_tllib_osc.sh:1-460

Purpose: Builds the TLib environment and applies local source repairs required by the OSC-tested dependency stack.

- Upstream handoff: Clones thuml/Transfer-Learning-Library and rewrites specific files under tllib/... to keep that checkout compatible with the validated OSC stack.

Top-level shell stages

```
question_7_transfer_learning/setup_tllib_osc.sh:2-3: !usr/bin/env bash
question_7_transfer_learning/setup_tllib_osc.sh:5-4: Prepare the repo-local TLib checkout used by the tracked Q7 wrapper. This
question_7_transfer_learning/setup_tllib_osc.sh:6-5: setup script also owns the small compatibility repairs that keep TLib usable
question_7_transfer_learning/setup_tllib_osc.sh:7-33: on the OSC-validated dependency stack.
question_7_transfer_learning/setup_tllib_osc.sh:35-42: Print one short compiler banner for diagnostics without failing setup.
question_7_transfer_learning/setup_tllib_osc.sh:44-43: Detect NVHPC-style compiler wrappers because Detectron2 builds more
question_7_transfer_learning/setup_tllib_osc.sh:45-50: reliably on OSC when GCC is selected explicitly instead.
question_7_transfer_learning/setup_tllib_osc.sh:52-51: Apply the local source edits that keep the checked-out TLib revision
question_7_transfer_learning/setup_tllib_osc.sh:53-261: compatible with the repo's validated torchvision / detectron2 stack.
question_7_transfer_learning/setup_tllib_osc.sh:263-262: Allow a relative target path at invocation time, then normalize it before the
question_7_transfer_learning/setup_tllib_osc.sh:264-275: rest of the script reuses the resolved repository location.
question_7_transfer_learning/setup_tllib_osc.sh:277-320: Clone once into external/ so this directory keeps only local scripts, data, and
results.
question_7_transfer_learning/setup_tllib_osc.sh:322-331: Capture the selected interpreter version before reusing or rebuilding the
environment.
question_7_transfer_learning/setup_tllib_osc.sh:333-340: Refuse to reuse an environment built with a different Python minor version.
question_7_transfer_learning/setup_tllib_osc.sh:342-345: Build the repository-local virtual environment that the helper scripts will reuse.
question_7_transfer_learning/setup_tllib_osc.sh:347-346: Install the baseline TLib requirements first, then add only the extra packages
```

Shell functions

compiler_banner at question_7_transfer_learning/setup_tllib_osc.sh:33-41. Category: local helper logic. Summary: Implements compiler_banner as a local helper logic helper.

compiler_looks_nvhpcc at question_7_transfer_learning/setup_tllib_osc.sh:42-49. Category: local helper logic. Summary: Implements compiler_looks_nvhpcc as a local helper logic helper.

apply_tllib_torchvision_compat at question_7_transfer_learning/setup_tllib_osc.sh:50-460. Category: local helper logic. Summary: Implements apply_tllib_torchvision_compat as a local helper logic helper.

Representative commands and side effects

```
set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
REPO_ROOT="$(cd "${SCRIPT_DIR}/.." && pwd)"
DEFAULT_REPO_DIR="${REPO_ROOT}/external/Transfer-Learning-Library"
REPO_URL="${REPO_URL:-https://github.com/thuml/Transfer-Learning-Library.git}"
REPO_DIR="{1:-${DEFAULT_REPO_DIR}}"
PYTHON_BIN="${PYTHON_BIN:-}"
```

INSTALL_TORCH="\${INSTALL_TORCH:-0}"

Not found in repo

- A target AP threshold that defines success for Q7 is not found in repo.
- A tracked provenance table tying each compatibility patch to a specific upstream TLLib commit or issue is not found in repo, even though the patched files and repair logic are present locally.

Appendix: Question Folder Inventory

Q3 Faster R-CNN Object Detection

question_3_faster_rcnn/README.md | type=markdown | lines=128
question_3_faster_rcnn/demo_fasterrcnn.py | type=python | lines=1107
question_3_faster_rcnn/download_models_fasterrcnn.sh | type=shell | lines=55
question_3_faster_rcnn/setup_fasterrcnn_osc.sh | type=shell | lines=164

Q4 YOLO11 and YOLOv12 Object Detection

question_4_yolo11_yolov12/README.md | type=markdown | lines=120
question_4_yolo11_yolov12/demo_yolo11.py | type=python | lines=492
question_4_yolo11_yolov12/demo_yolov12.py | type=python | lines=509
question_4_yolo11_yolov12/setup_yolo11_osc.sh | type=shell | lines=94
question_4_yolo11_yolov12/setup_yolov12_osc.sh | type=shell | lines=129

Q5 YOLO11 Semantic Segmentation on iSAID

question_5_semantic_segmentation/README.md | type=markdown | lines=234
question_5_semantic_segmentation/bootstrap_isaid_seg.py | type=python | lines=293
question_5_semantic_segmentation/demo_yolo_segmentation.py | type=python | lines=777
question_5_semantic_segmentation/q5_seg_common.py | type=python | lines=927
question_5_semantic_segmentation/setup_yolo11_osc.sh | type=shell | lines=216
question_5_semantic_segmentation/train_isaid_seg.py | type=python | lines=336

Q6 Directional Convolution Filters

question_6_convolution_filters/README.md | type=markdown | lines=73
question_6_convolution_filters/convolution_filter_library.py | type=python | lines=188
question_6_convolution_filters/demo_convolution_filters.py | type=python | lines=142
question_6_convolution_filters/demo_convolution_filters_image.py | type=python | lines=363

Q7 TLLib VOC-to-Clipart Transfer Learning

question_7_transfer_learning/README.md | type=markdown | lines=219
question_7_transfer_learning/demo_tllib_object_detection.py | type=python | lines=1741
question_7_transfer_learning/run_tllib_osc.sh | type=shell | lines=96
question_7_transfer_learning/setup_tllib_osc.sh | type=shell | lines=460